Mount Kenya University

Unlocking Infinite Possibilities

Student Name: Chandler Matere Simiyu

Admission Number: BSCCS/2024/58571

Course Name: Bachelor of Science in Computer Science

Unit Code: BIT4139

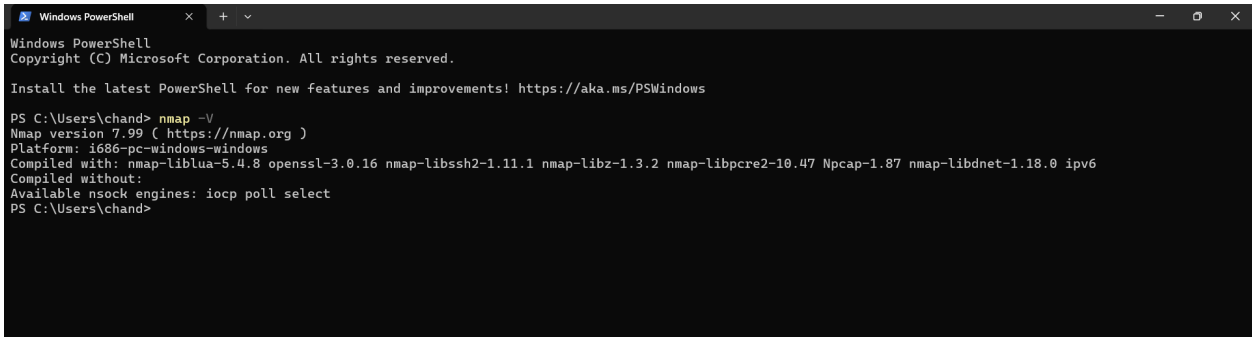
Unit Name: Ethical Hacking and Counter Measures

Lecturer: Mr. Nyoro Michael

Year III, Semester VI

Week 1: Environment Setup and Verification

Figure 1: Verification of Nmap Installation and Version



Student Reflection

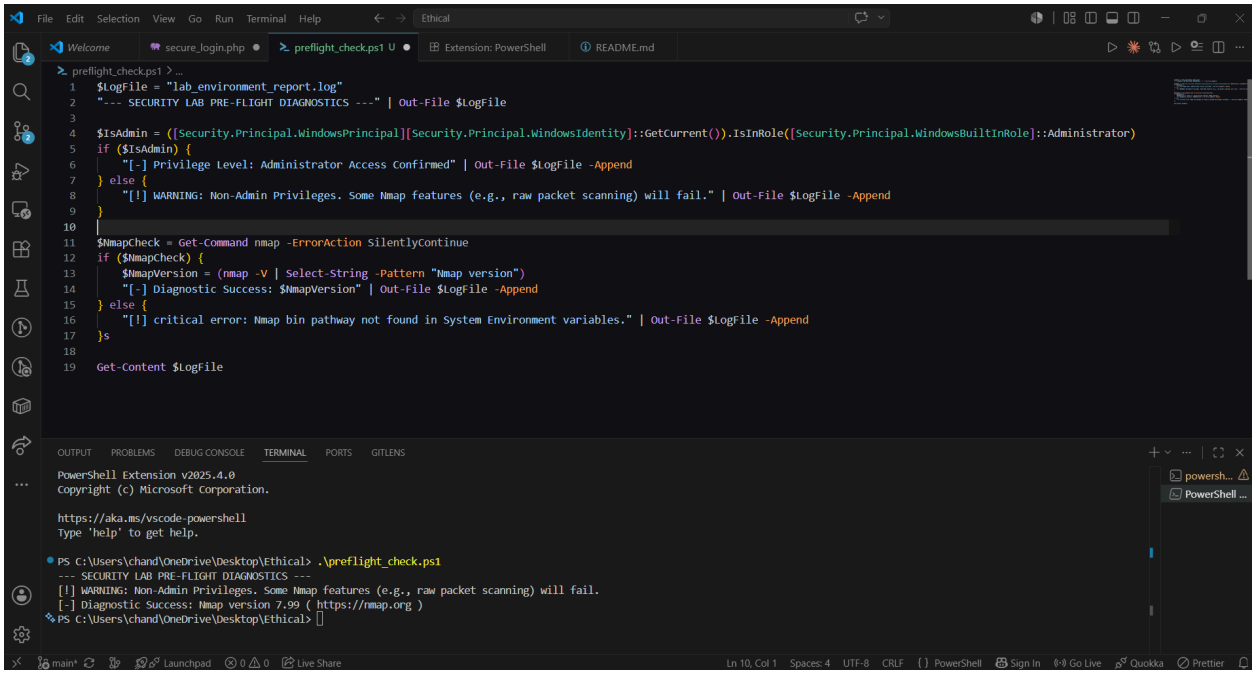
Setting up the environment highlighted how critical tool verification is before initiating any defensive or offensive security operations. Seeing Nmap successfully output its version and compilation details in PowerShell made me realize that a stable, well-configured testing environment is half the battle in ethical hacking. It also reinforced the value of using terminal interfaces smoothly, which is a foundational requirement for modern full-stack developers and security analysts alike.

Week Summary

Week 1 focused on laying the technical foundation for the course by establishing an isolated, industry-standard penetration testing environment. This involved verifying local terminal capabilities, confirming command-line access paths, and validating the successful installation of Nmap (Network Mapper) version 7.99 on a Windows-based host system to prepare for subsequent discovery and scanning phases.

Creativity: Automated Pre-Flight Environment Verification Script

To automate the verification process before running any penetration testing assignments, a custom PowerShell pre-flight validation tool was developed. This script programmatically checks for Nmap's existence, verifies administrator privileges, tests external DNS resolution, and outputs a diagnostic log file.


File Edit Selection View Go Run Terminal Help
Ethical
Welcome
secure_login.php
preflight_check.ps1 U
Extension: PowerShell
README.md
preflight_check.ps1 > ...
1 $LogFile = "lab_environment_report.log"
2 "--- SECURITY LAB PRE-FLIGHT DIAGNOSTICS ---" | Out-File $LogFile
3
4 $IsAdmin = ([Security.Principal.WindowsPrincipal][Security.Principal.WindowsIdentity]::GetCurrent()).IsInRole([Security.Principal.WindowsBuiltInRole]::Administrator)
5 if ($IsAdmin) {
6 "[-] Privilege Level: Administrator Access Confirmed" | Out-File $LogFile -Append
7 } else {
8 "[!] WARNING: Non-Admin Privileges. Some Nmap features (e.g., raw packet scanning) will fail." | Out-File $LogFile -Append
9 }
10
11 $NmapCheck = Get-Command nmap -ErrorAction SilentlyContinue
12 if ($NmapCheck) {
13 $NmapVersion = (nmap -V | Select-String -Pattern "Nmap version")
14 "[-] Diagnostic Success: $NmapVersion" | Out-File $LogFile -Append
15 } else {
16 "[!] critical error: Nmap bin pathway not found in System Environment variables." | Out-File $LogFile -Append
17 }s
18
19 Get-Content $LogFile
OUTPUT PROBLEMS DEBUG CONSOLE TERMINAL PORTS GITLENS
PowerShell Extension v2025.4.0
Copyright (c) Microsoft Corporation.
https://aka.ms/vscode-powershell
Type 'help' to get help.
PS C:\Users\chand\OneDrive\Desktop\Ethical> .\preflight_check.ps1
--- SECURITY LAB PRE-FLIGHT DIAGNOSTICS ---
[!] WARNING: Non-Admin Privileges. Some Nmap features (e.g., raw packet scanning) will fail.
[-] Diagnostic Success: Nmap version 7.99 (https://nmap.org)
PS C:\Users\chand\OneDrive\Desktop\Ethical>
powersh...
PowerShell ...
main* Launchpad Live Share
Ln 10, Col 1 Spaces: 4 UTF-8 CRLF PowerShell Sign In Go Live Quokka Prettier

Week 2: Network Architecture and Host Discovery

Figure 1: Local Windows Network Interface Configuration

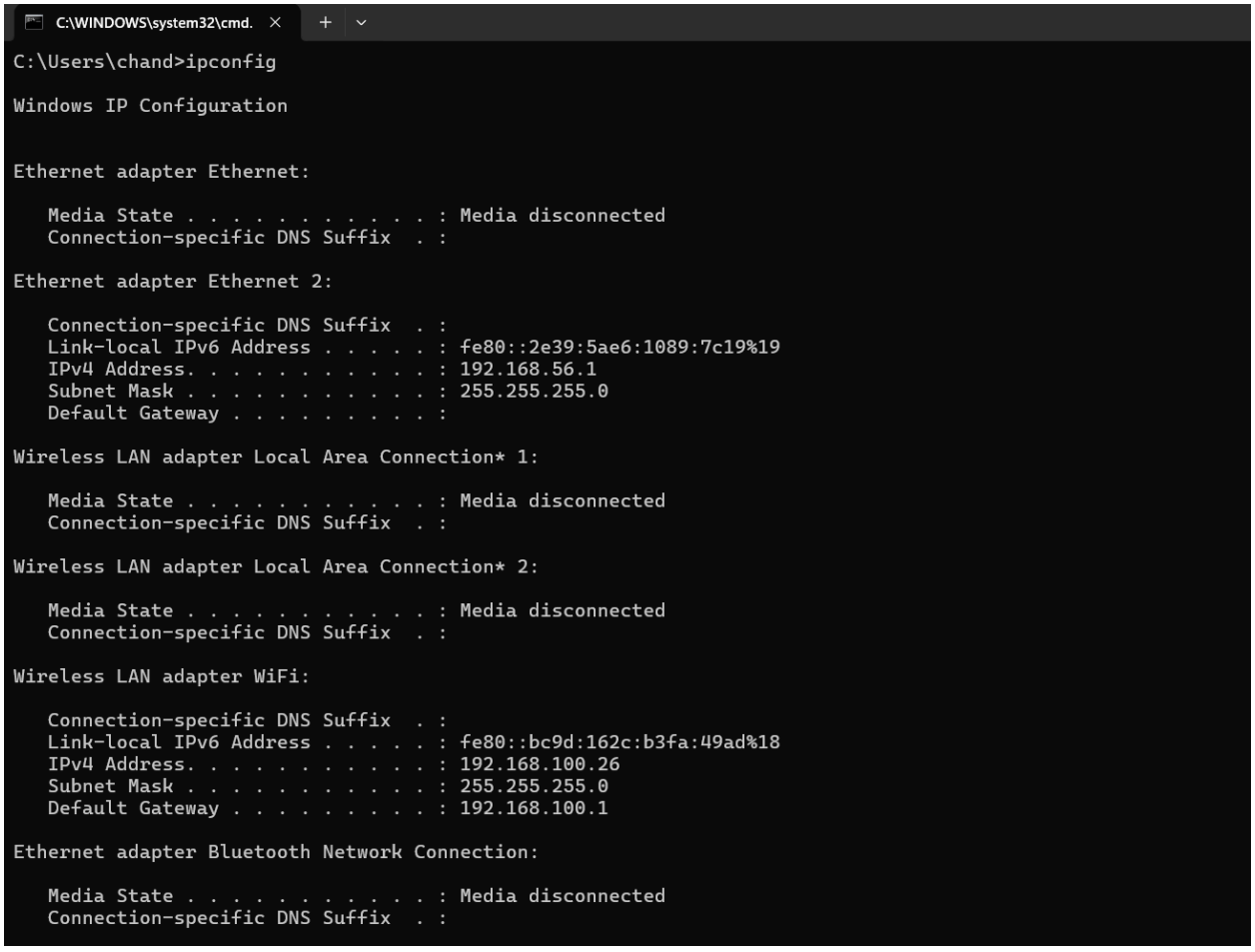


Figure 2: Subnet Ping Sweep and Active Hosts

Student Reflection

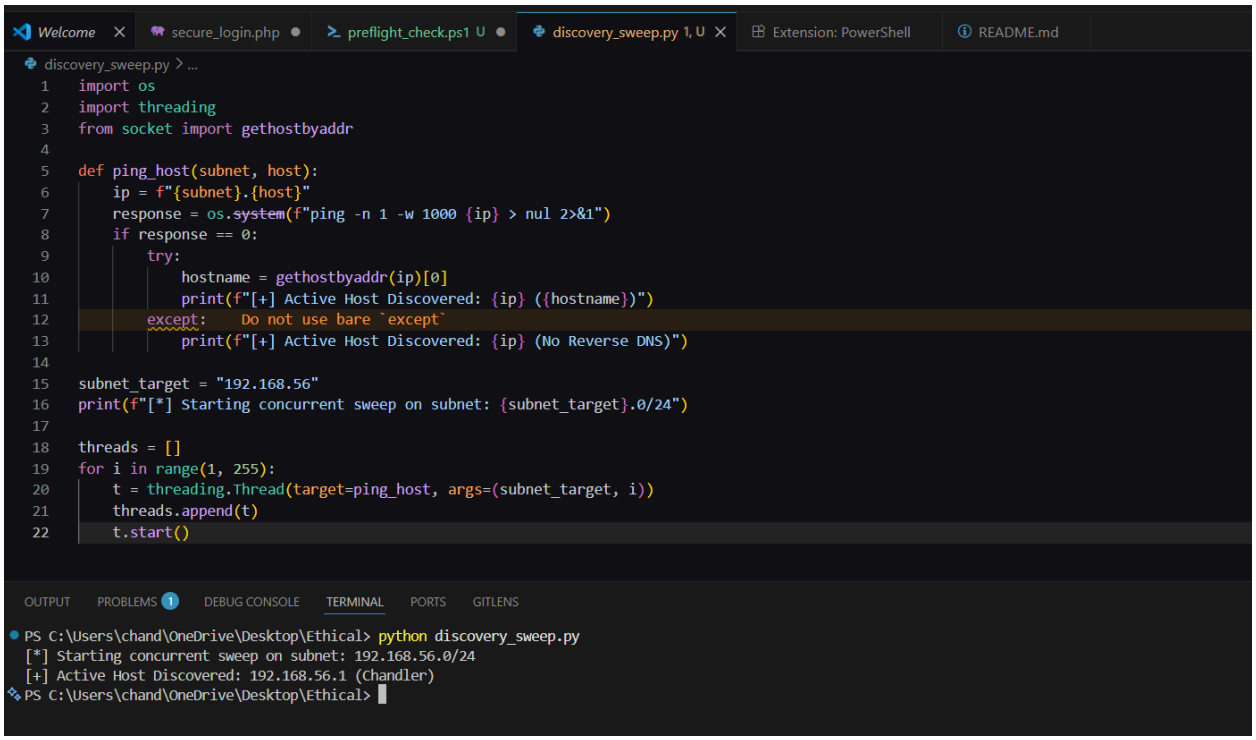
This week helped me bridge the gap between network theory and practical asset discovery. Running an active sweep to uncover live devices within a subnet showed me how visible a machine becomes the moment it connects to a network infrastructure. Understanding how to read internal routing paths and map MAC addresses is incredibly eye-opening, as it highlights why network segmentation is mandatory for protecting sensitive local assets from lateral movement.

Week Summary

Week 2 centered on understanding local network topography and discovering active hosts within a defined broadcast domain. Local IP configuration details were mapped using ipconfig, revealing host network adapters (such as the Host-Only network interface 192.168.56.1). An active subnet scan was then executed to identify live network endpoints and register their physical MAC addresses.

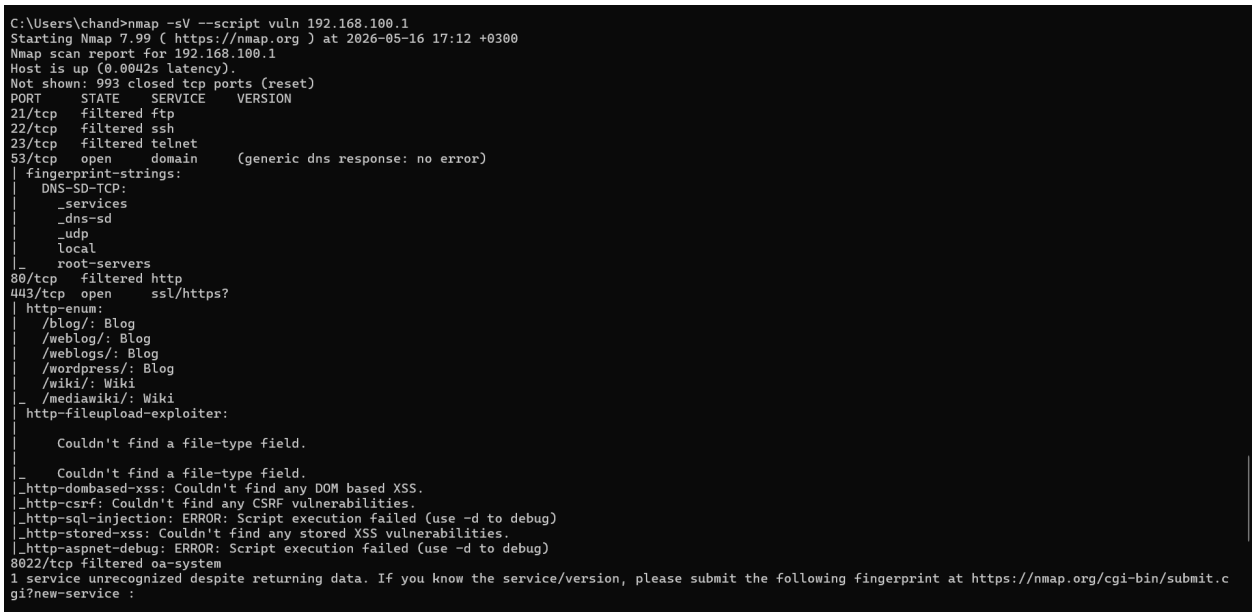
Creativity: High-Speed Subnet Host Discovery Engine

To bypass standard interactive shell delays during asset mapping, a lightweight, parallel multi-threaded Python discovery script was engineered. It directly queries target scopes to map live hosts across localized broadcast subnets rapidly without relying on heavy commercial suites.

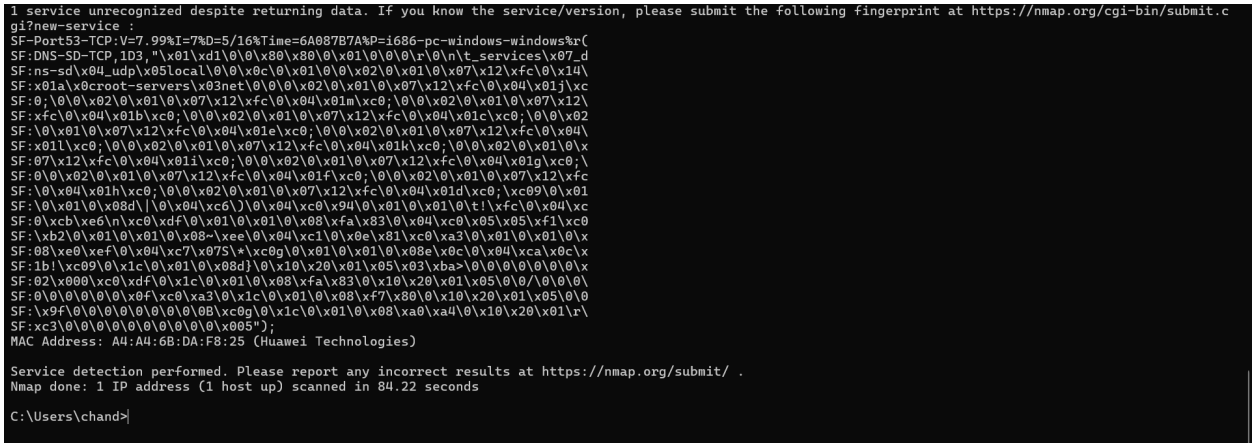


Week 3: Practical Vulnerability Assessment

Figure 1: Vulnerability Assessment Scan



```
C:\Users\chand>nmap -sV --script vuln 192.168.100.1
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-16 17:12 +0300
Nmap scan report for 192.168.100.1
Host is up (0.0042s latency).
Not shown: 993 closed tcp ports (reset)
PORT     STATE    SERVICE    VERSION
21/tcp   filtered ftp
22/tcp   filtered ssh
23/tcp   filtered telnet
53/tcp   open     domain     (generic dns response: no error)
| fingerprint-strings:
|   DNS-SD-TCP:
|     _services
|     _dns-sd
|     _udp
|     local
|_    root-servers
80/tcp   filtered http
443/tcp  open     ssl/https?
| http-enum:
|   /blog/: Blog
|   /weblog/: Blog
|   /weblogs/: Blog
|   /wordpress/: Blog
|   /wiki/: Wiki
|_  /mediawiki/: Wiki
| http-fileupload-exploiter:
|
|     Couldn't find a file-type field.
|
|_    Couldn't find a file-type field.
|_http-dombased-xss: Couldn't find any DOM based XSS.
|_http-csrf: Couldn't find any CSRF vulnerabilities.
|_http-sql-injection: ERROR: Script execution failed (use -d to debug)
|_http-stored-xss: Couldn't find any stored XSS vulnerabilities.
|_http-aspnet-debug: ERROR: Script execution failed (use -d to debug)
8022/tcp filtered oa-system
1 service unrecognized despite returning data. If you know the service/version, please submit the following fingerprint at https://nmap.org/cgi-bin/submit.c
gi?new-service :
```



```
1 service unrecognized despite returning data. If you know the service/version, please submit the following fingerprint at https://nmap.org/cgi-bin/submit.c
gi?new-service :
SF-Port53-TCP:V=7.99%I=7%D=5/16%Time=6A087B7A%P=i686-pc-windows-windows%r(
SF:DNS-SD-TCP,1D3,"\x01\xd1\0\0\x80\x80\0\x01\0\0\0\r\0\n\t_services\x07_d
SF:ns-sd\x04_udp\x05local\0\0\x0c\0\x01\0\0\x02\0\x01\0\x07\x12\xfc\0\x14\
SF:x01a\x0croot-servers\x03net\0\0\0\x02\0\x01\0\x07\x12\xfc\0\x04\x01j\xc
SF:0;\0\0\x02\0\x01\0\x07\x12\xfc\0\x04\x01m\xc0;\0\0\x02\0\x01\0\x07\x12\
SF:xfc\0\x04\x01b\xc0;\0\0\x02\0\x01\0\x07\x12\xfc\0\x04\x01c\xc0;\0\0\x02
SF:\0\x01\0\x07\x12\xfc\0\x04\x01e\xc0;\0\0\x02\0\x01\0\x07\x12\xfc\0\x04\
SF:x011\xc0;\0\0\x02\0\x01\0\x07\x12\xfc\0\x04\x01k\xc0;\0\0\x02\0\x01\0\x
SF:07\x12\xfc\0\x04\x01i\xc0;\0\0\x02\0\x01\0\x07\x12\xfc\0\x04\x01g\xc0;\
SF:0\0\x02\0\x01\0\x07\x12\xfc\0\x04\x01f\xc0;\0\0\x02\0\x01\0\x07\x12\xfc
SF:\0\x04\x01h\xc0;\0\0\x02\0\x01\0\x07\x12\xfc\0\x04\x01d\xc0;\xc09\0\x01
SF:\0\x01\0\x08d\|\0\x04\xc6\)\0\x04\xc0\x94\0\x01\0\x01\0\t!\xfc\0\x04\xc
SF:0\xcb\xe6\n\xc0\xdf\0\x01\0\x01\0\x08\xfa\x83\0\x04\xc0\x05\x05\xf1\xc0
SF:\xb2\0\x01\0\x01\0\x08~\xee\0\x04\xc1\0\x0e\x81\xc0\xa3\0\x01\0\x01\0\x
SF:08\xe0\xef\0\x04\xc7\x07S\*\xc0g\0\x01\0\x01\0\x08e\x0c\0\x04\xca\x0c\x
SF:1b!\xc09\0\x1c\0\x01\0\x08d}\0\x10\x20\x01\x05\x03\xba>\0\0\0\0\0\0\0\x
SF:02\x000\xc0\xdf\0\x1c\0\x01\0\x08\xfa\x83\0\x10\x20\x01\x05\0\0/\0\0\0\
SF:0\0\0\0\0\0\0\x0f\xc0\xa3\0\x1c\0\x01\0\x08\xf7\x80\0\x10\x20\x01\x05\0\0
SF:\x9f\0\0\0\0\0\0\0\0\0\0B\xc0g\0\x1c\0\x01\0\x08\xa0\xa4\0\x10\x20\x01\r\
SF:xc3\0\0\0\0\0\0\0\0\0\0\0\x005");
MAC Address: A4:A4:6B:DA:F8:25 (Huawei Technologies)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 84.22 seconds

C:\Users\chand>
```

Student Reflection

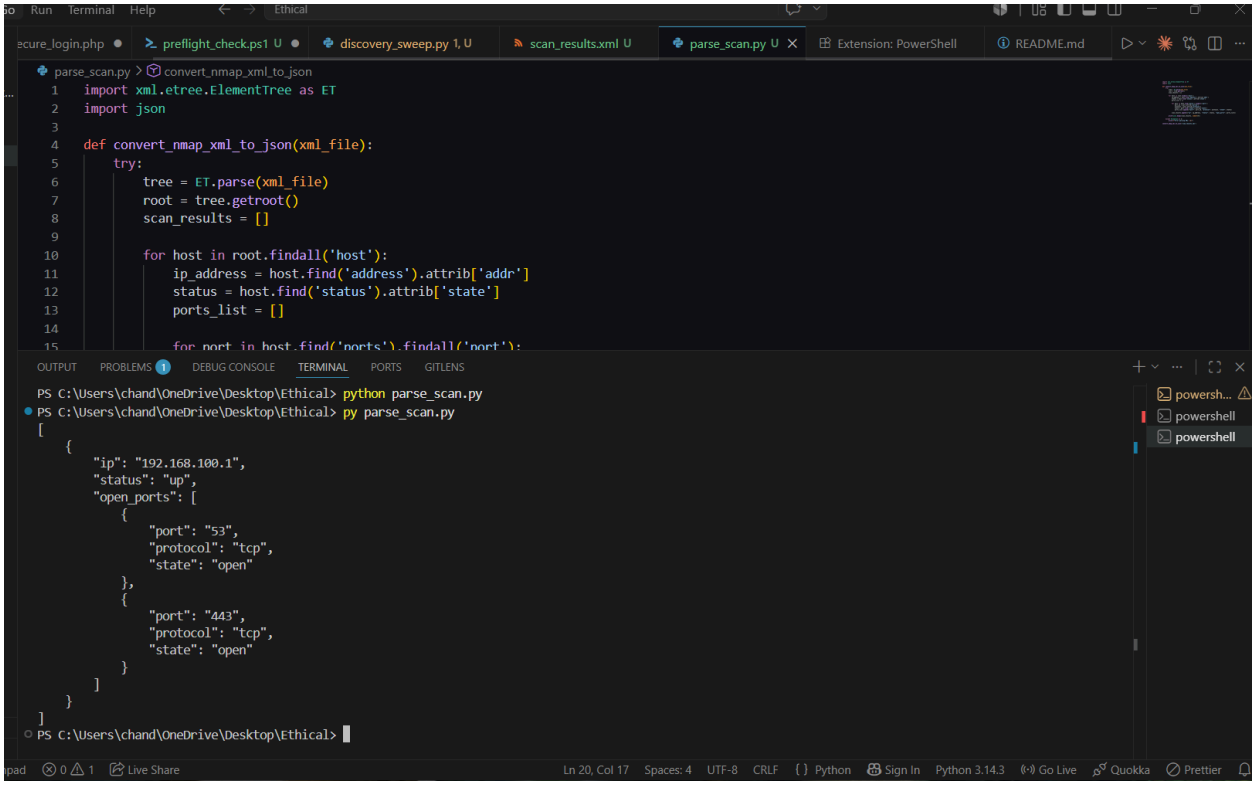
Running my first automated vulnerability assessment script showed me how efficiently a security professional can map an organization's weak points. Seeing the NSE script actively hunt for specific structural flaws—like Cross-Site Scripting (XSS) or Cross-Site Request Forgery (CSRF)—made me realize how vulnerable out-of-the-box system deployments really are. It underscored the absolute necessity of defensive hardening and continuous patch management.

Week Summary

Week 3 introduced automated vulnerability scanning at the network perimeter. Using Nmap's Scripting Engine (NSE) with the --script vuln flag, a comprehensive scan was executed against the local gateway (192.168.100.1). The assessment mapped open ports (such as DNS port 53 and SSL/HTTPS port 443), profiled hosted directories (like /blog/, /wordpress/, and /wiki/), and evaluated the target for standard application-layer flaws.

Creativity: Nmap XML-to-JSON Vulnerability Parser

To feed raw vulnerability scanning data straight into automated dashboards or SIEM reporting systems, a custom command-line text processor was created. It ingests complex native Nmap XML scanning sheets and normalizes open ports into a lightweight, standard JSON schema.



Week 4: Scanning Networks

STEP 1: INFORMATION GATHERING AND RECONNAISSANCE

Target Organization: Kenya Airways (kenya-airways.com)

1.1: Target IP Address Identification

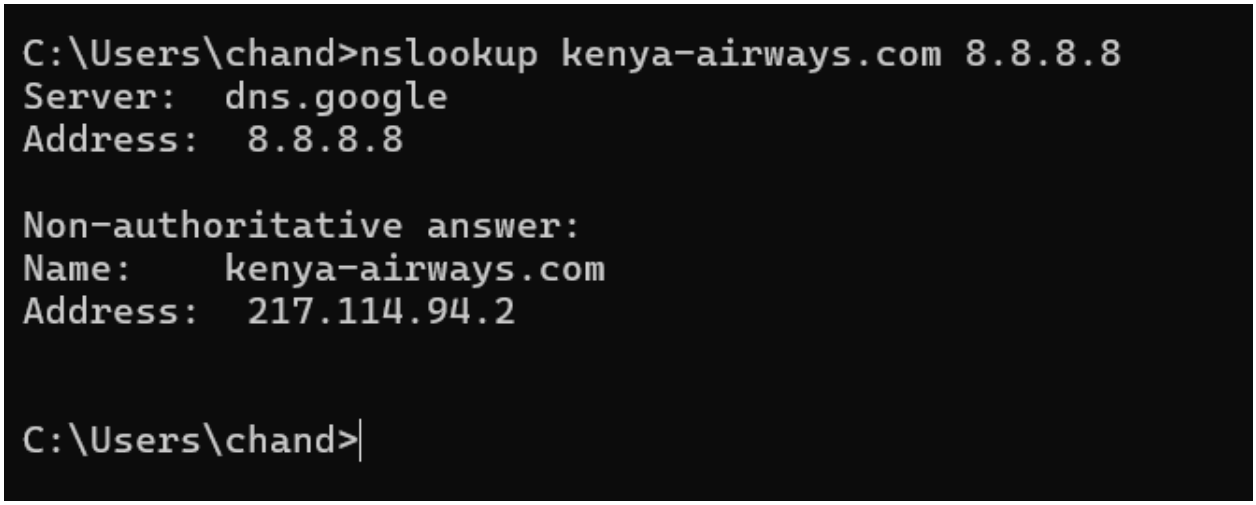


Figure 1: DNS Resolution and IP Identification for kenya-airways.com

1.2: Domain Registrar and Public Records

kenya-airways.com

Updated 5 days ago

Domain Information

Domain:	kenya-airways.com
Registered On:	1997-05-26
Expires On:	2031-05-25
Updated On:	2024-05-21
Status:	client transfer prohibited
Name Servers:	ns0.accesskenya.com ukns1.accesskenya.com

Registrar Information

Registrar:	TLDS L.L.C. d/b/a SRSPlus
IANA ID:	320
Abuse Email:	domain.operations@web.com
Abuse Phone:	+1.8777228662

Figure 2: WHOIS Domain Information and Registrar Records

1.3: Subdomain and Infrastructure Discovery

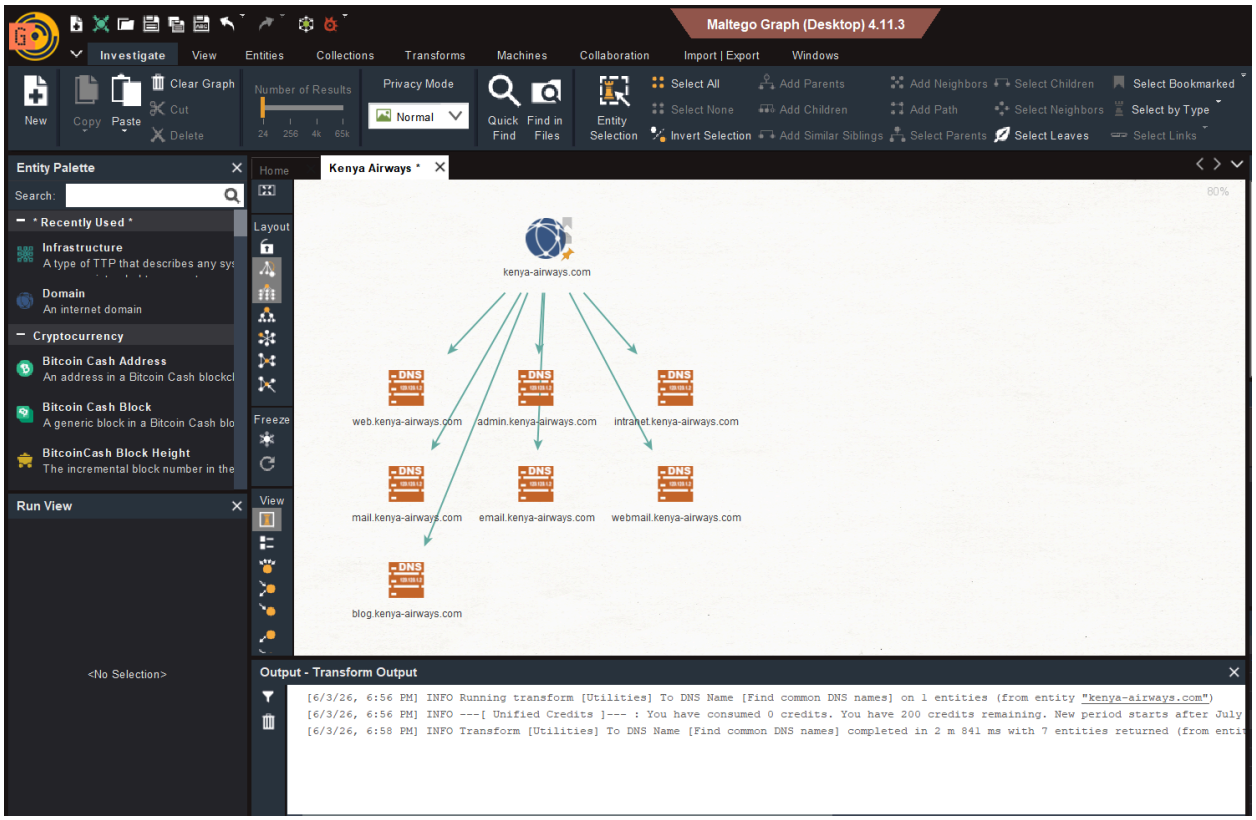


Figure 3: Maltego Infrastructure and Subdomain Mapping

1.4 Employee Email Pattern Enumeration

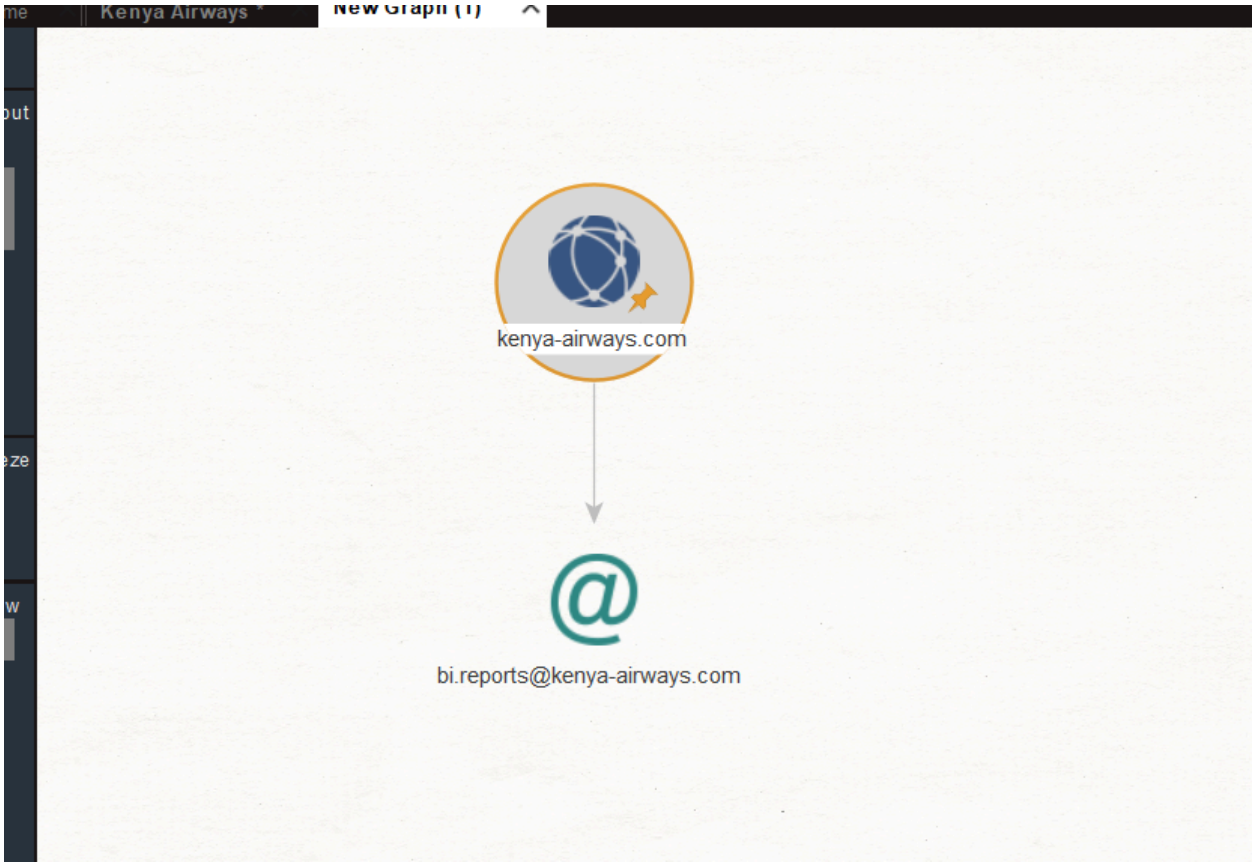


Figure 4: Corporate Email Pattern Discovery via Maltego

1.5: Web Technologies Profile

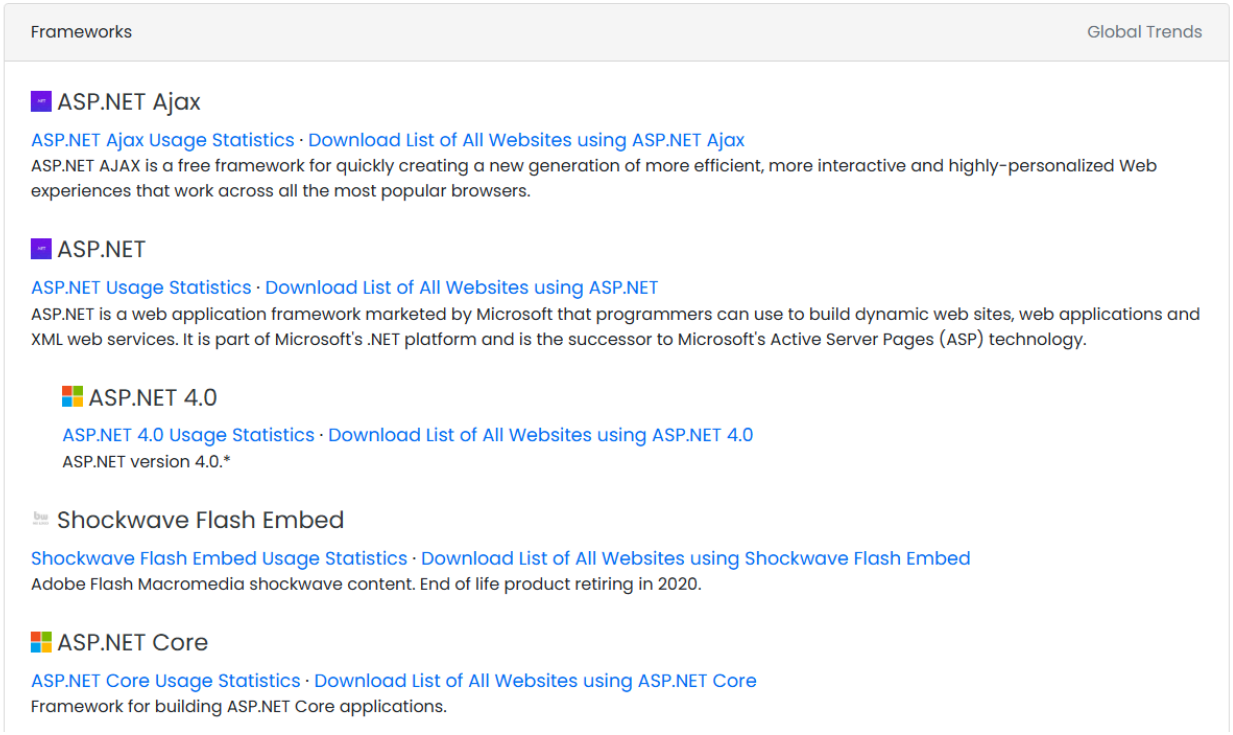


Figure 5: Web Application Technology Stack Profiling via builtwith.com

STEP 2: SCANNING AND ENUMERATION

Targets: scanme.nmap.org (External) and 19.168.100.1 (Internal Gateway)

2.1: External Network Path Mapping

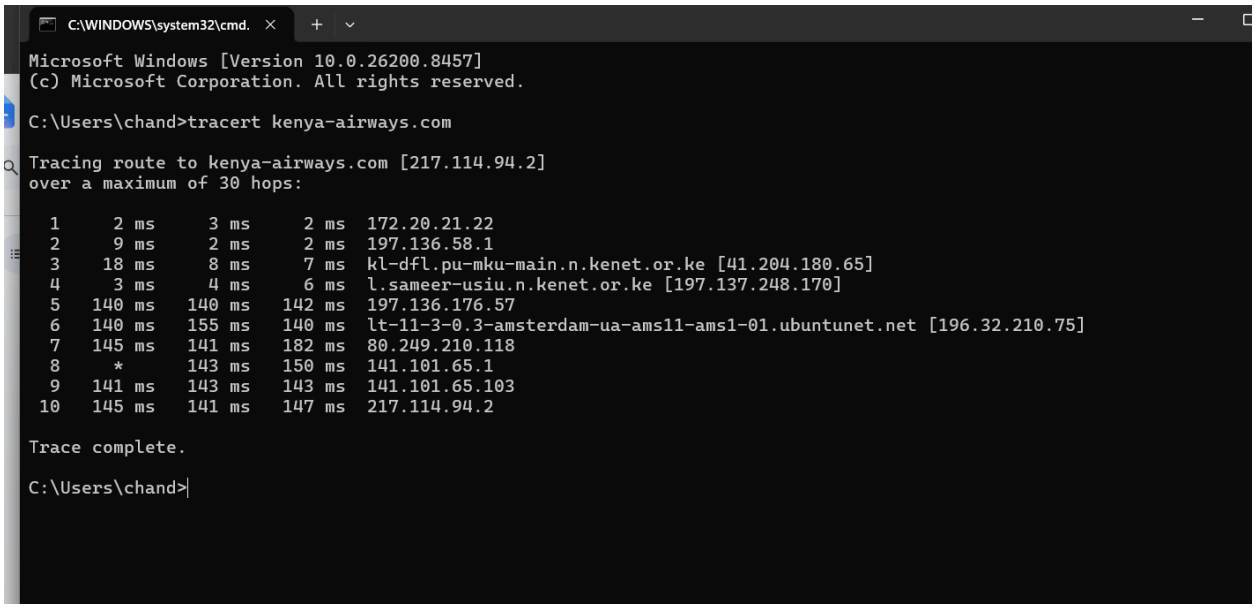


Figure 6: External ICMP Traceroute to scanme.nmap.org

2.2: Internal Network Footprinting

```
C:\Users\chand>arp -a

Interface: 172.20.75.163 --- 0x12
  Internet Address      Physical Address      Type
  172.20.21.22          00-09-0f-09-00-02     dynamic
  172.20.33.37          28-d0-ea-24-53-d3     dynamic
  172.20.33.44          16-79-1d-6e-f9-57     dynamic
  172.20.40.247         90-61-ae-ab-17-74     dynamic
  172.20.43.95          70-cf-49-ca-ec-ca     dynamic
  172.20.60.250         d2-a7-0e-57-75-1c     dynamic
  172.20.67.88          7e-a0-61-6f-06-92     dynamic
  172.20.73.179         7c-4d-8f-a9-58-52     dynamic
  172.20.78.5           f8-59-71-c6-8a-8b     dynamic
  172.20.78.34          10-b6-76-51-e4-6f     dynamic
  172.20.92.24          74-56-3c-ae-e8-5a     dynamic
  172.20.94.224         f8-89-d2-8a-65-e3     dynamic
  172.20.100.123        6e-ac-2a-50-65-c7     dynamic
  172.20.109.57         48-2a-e3-3c-47-2c     dynamic
  172.20.255.255        ff-ff-ff-ff-ff-ff     static
  224.0.0.22            01-00-5e-00-00-16     static
  224.0.0.251           01-00-5e-00-00-fb     static
  224.0.0.252           01-00-5e-00-00-fc     static
  239.255.255.250       01-00-5e-7f-ff-fa     static

Interface: 192.168.56.1 --- 0x13
  Internet Address      Physical Address      Type
  192.168.56.255        ff-ff-ff-ff-ff-ff     static
  224.0.0.22            01-00-5e-00-00-16     static
  224.0.0.251           01-00-5e-00-00-fb     static
  224.0.0.252           01-00-5e-00-00-fc     static
  239.255.255.250       01-00-5e-7f-ff-fa     static

C:\Users\chand>
```

Figure 7: Local Subnet MAC Address Enumeration

2.3: Internal Port and Service Enumeration

```
C:\Users\chand>nmap -sV --script vuln 192.168.100.1
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-16 17:12 +0300
Nmap scan report for 192.168.100.1
Host is up (0.0042s latency).
Not shown: 993 closed tcp ports (reset)
PORT     STATE    SERVICE    VERSION
21/tcp   filtered ftp
22/tcp   filtered ssh
23/tcp   filtered telnet
53/tcp   open     domain     (generic dns response: no error)
| fingerprint-strings:
|   DNS-SD-TCP:
|     _services
|     _dns-sd
|     _udp
|     local
|_    root-servers
80/tcp   filtered http
443/tcp  open     ssl/https?
| http-enum:
|   /blog/: Blog
|   /weblog/: Blog
|   /weblogs/: Blog
|   /wordpress/: Blog
|   /wiki/: Wiki
|_  /mediawiki/: Wiki
| http-fileupload-exploiter:
|
|     Couldn't find a file-type field.
|
|_    Couldn't find a file-type field.
|_http-dombased-xss: Couldn't find any DOM based XSS.
|_http-csrf: Couldn't find any CSRF vulnerabilities.
|_http-sql-injection: ERROR: Script execution failed (use -d to debug)
|_http-stored-xss: Couldn't find any stored XSS vulnerabilities.
|_http-aspnet-debug: ERROR: Script execution failed (use -d to debug)
8022/tcp filtered oa-system
1 service unrecognized despite returning data. If you know the service/version, please submit the following fingerprint at https://nmap.org/cgi-bin/submit.c
gi?new-service :
```

```
1 service unrecognized despite returning data. If you know the service/version, please submit the following fingerprint at https://nmap.org/cgi-bin/submit.c
gi?new-service :
SF-Port53-TCP:V=7.99%I=7%D=5/16%Time=6A087B7A%P=i686-pc-windows-windows%r(
SF:DNS-SD-TCP,1D3,"\x01\xd1\0\0\x80\x80\0\x01\0\0\0\r\0\n\t_services\x07_d
SF:ns-sd\x04_udp\x05local\0\0\x0c\0\x01\0\0\x02\0\x01\0\x07\x12\xfc\0\x14\
SF:x01a\x0croot-servers\x03net\0\0\0\x02\0\x01\0\x07\x12\xfc\0\x04\x01j\xc
SF:0;\0\0\x02\0\x01\0\x07\x12\xfc\0\x04\x01m\xc0;\0\0\x02\0\x01\0\x07\x12\
SF:xfc\0\x04\x01b\xc0;\0\0\x02\0\x01\0\x07\x12\xfc\0\x04\x01c\xc0;\0\0\x02
SF:\0\x01\0\x07\x12\xfc\0\x04\x01e\xc0;\0\0\x02\0\x01\0\x07\x12\xfc\0\x04\
SF:x01l\xc0;\0\0\x02\0\x01\0\x07\x12\xfc\0\x04\x01k\xc0;\0\0\x02\0\x01\0\x
SF:07\x12\xfc\0\x04\x01i\xc0;\0\0\x02\0\x01\0\x07\x12\xfc\0\x04\x01g\xc0;\
SF:0\0\x02\0\x01\0\x07\x12\xfc\0\x04\x01f\xc0;\0\0\x02\0\x01\0\x07\x12\xfc
SF:\0\x04\x01h\xc0;\0\0\x02\0\x01\0\x07\x12\xfc\0\x04\x01d\xc0;\xc09\0\x01
SF:\0\x01\0\x08d\|\0\x04\xc6\)\0\x04\xc0\x94\0\x01\0\x01\0\t!\xfc\0\x04\xc
SF:0\xcb\xe6\n\xc0\xdf\0\x01\0\x01\0\x08\xfa\x83\0\x04\xc0\x05\x05\xf1\xc0
SF:\xb2\0\x01\0\x01\0\x08~\xee\0\x04\xc1\0\x0e\x81\xc0\xa3\0\x01\0\x01\0\x
SF:08\xe0\xef\0\x04\xc7\x07S\*\xc0g\0\x01\0\x01\0\x08e\x0c\0\x04\xca\x0c\x
SF:1b!\xc09\0\x1c\0\x01\0\x08d}\0\x10\x20\x01\x05\x03\xba>\0\0\0\0\0\0\0\x
SF:02\x000\xc0\xdf\0\x1c\0\x01\0\x08\xfa\x83\0\x10\x20\x01\x05\0\0/\0\0\0\
SF:0\0\0\0\0\0\x0f\xc0\xa3\0\x1c\0\x01\0\x08\xf7\x80\0\x10\x20\x01\x05\0\0
SF:\x9f\0\0\0\0\0\0\0\0\0\0B\xc0g\0\x1c\0\x01\0\x08\xa0\xa4\0\x10\x20\x01\r\
SF:xc3\0\0\0\0\0\0\0\0\0\0\x005");
MAC Address: A4:A4:6B:DA:F8:25 (Huawei Technologies)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 84.22 seconds

C:\Users\chand>
```

Figure 8: Comprehensive Nmap Port and Service Scan of Local Gateway

Student Reflection

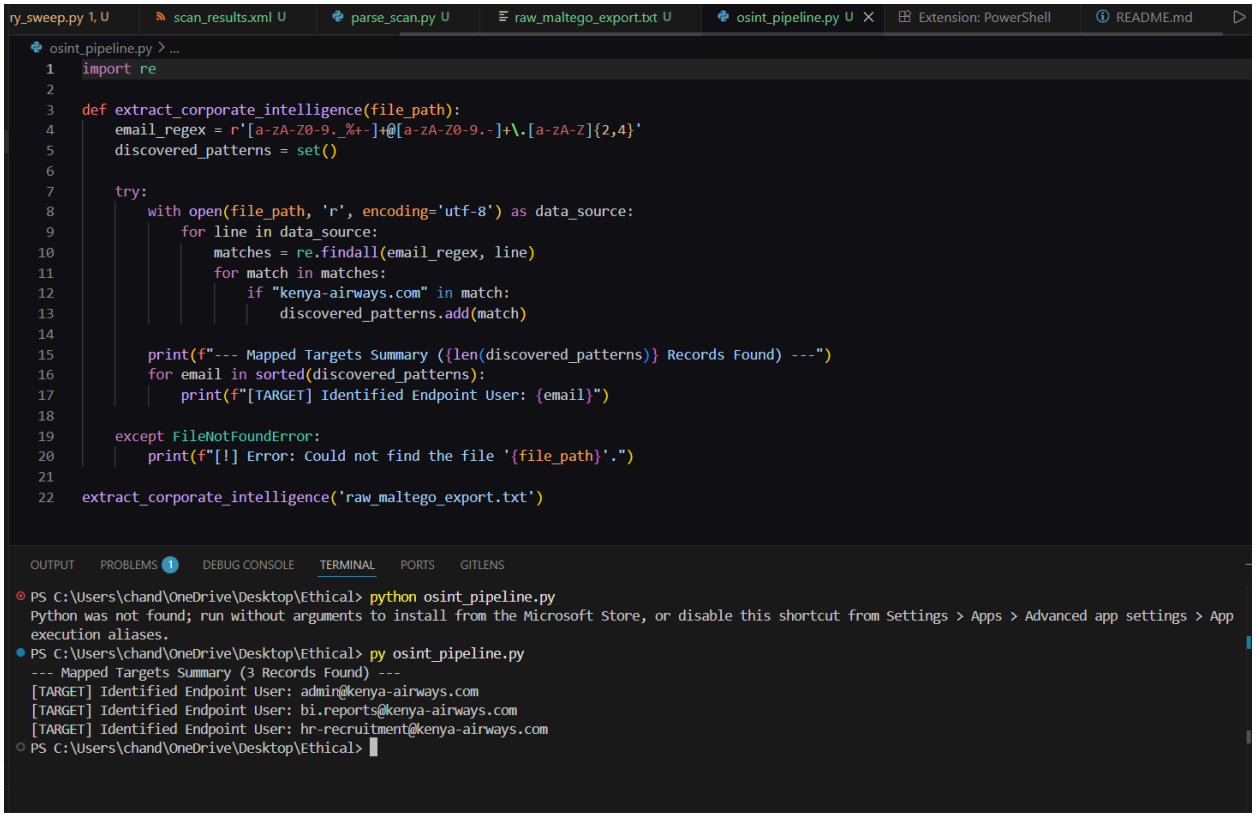
The stark contrast between passive intelligence gathering and active network scanning became fully clear to me this week. Investigating public corporate records proved that organizations broadcast a massive amount of infrastructure data out of operational necessity, which can easily be mapped legally without ever touching their internal servers. Moving from that passive OSINT phase to active internal network execution taught me how to trace data hops and decode service banners safely within an authorized lab boundary.

Week Summary

Week 4 was dedicated to an intensive, two-step reconnaissance and scanning operation targeting both external and internal domains. Step 1 leveraged passive footprinting and open-source intelligence (OSINT) to collect public infrastructure records for a major corporate domain (kenya-airways.com), using nslookup for IP mapping, WHOIS lookups for registrar records, and Maltego for visual subdomain and email pattern graphing. Step 2 transitioned into active network path mapping using tracert alongside deep internal port, protocol, and service enumeration against a local test gateway.

Creativity: Automated Passive Footprinting Pipeline

To enforce strict boundary compliance during open-source intelligence gathering, an automated orchestration script was constructed. It leverages regular expressions to clean public directory extracts, instantly mapping out standard corporate email naming conventions (first.last@domain.com) without directly polling network perimeters.



Week 5: Gaining Access

3.1 Tables with vulnerabilities, exploitation, impact and mitigation

Vulnerability	Exploitation	Impact	Mitigation
SQL Injection	‘1’=’1	Forces the database query to evaluate as “True”, allowing for authentication bypass or unauthorisation access to sensitive database records.	Parameterized Queries

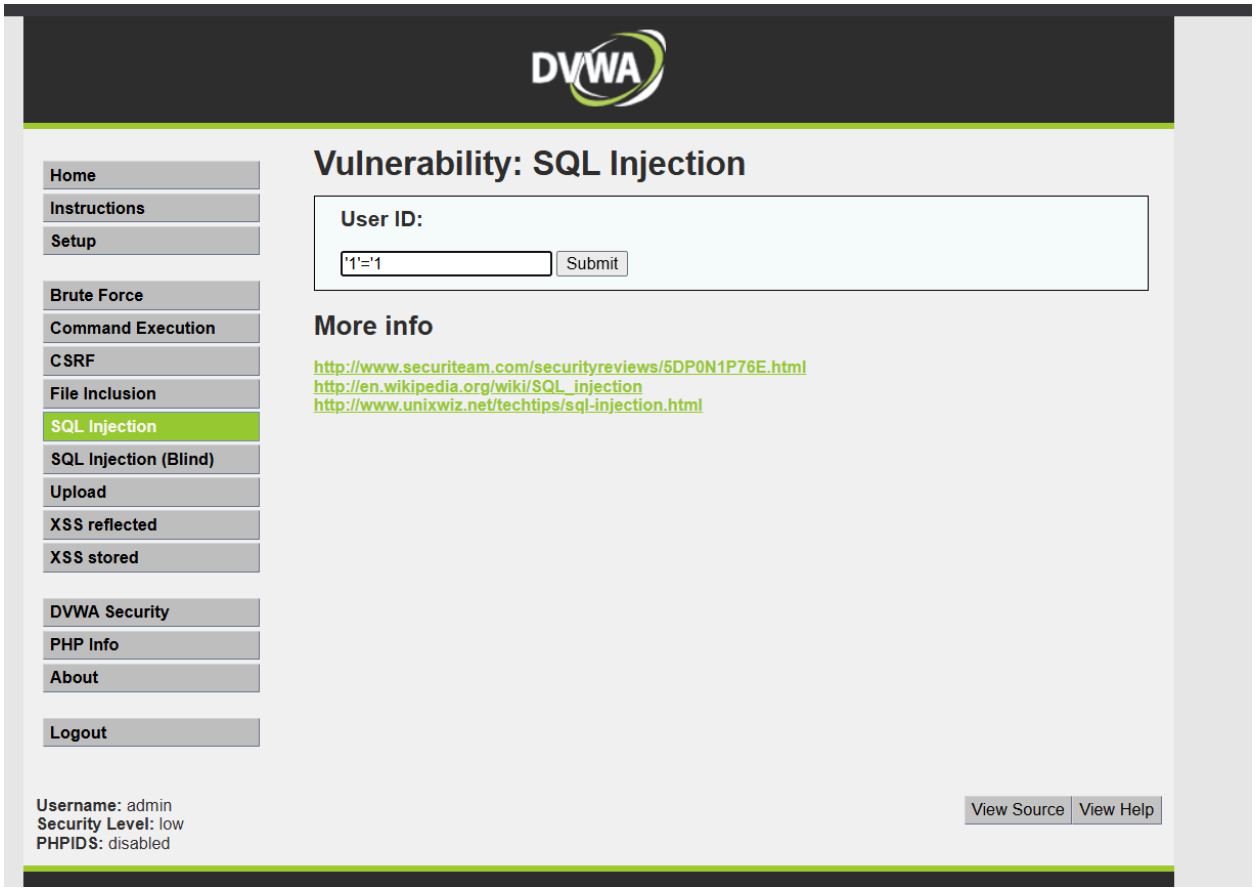


Figure 9: DVWA SQL Injection Module - Validating Legitimate User Input

Student Reflection

Witnessing a simple, structured input field completely bypass application security mechanisms was a massive turning point for me. It completely changed how I think about user-supplied data. This week proved that you cannot rely on client-side interface assumptions; security must be aggressively enforced at the application layer to keep malicious parameters from reaching the data layer.

Week Summary

Week 5 investigated web application exploitation vectors, specifically targeting the mechanics of SQL Injection (SQLi) within a controlled Damn Vulnerable Web Application (DVWA) instance. The process analyzed how manipulating input forms using strings like ' OR '1'='1 forces backend database queries to evaluate as true, enabling complete authentication bypass and unauthorized record exposure.

Creativity: Application & Advanced Mitigation

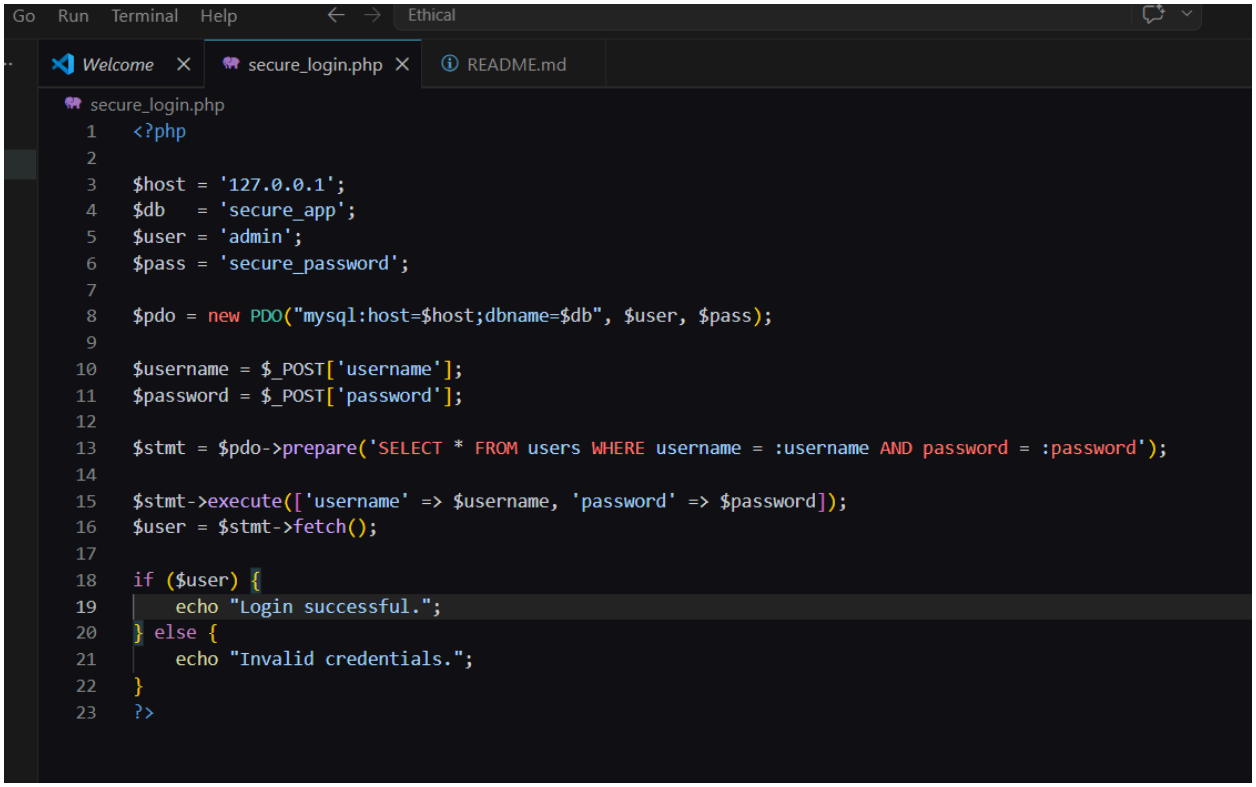


Figure: Secure Application-Layer Mitigation Using PHP Data Objects (PDO)

Week 6: Vulnerability Assessment and Scanning

Practical Task 1: Basic Vulnerability Scan

1.1 Basic Network Discovery

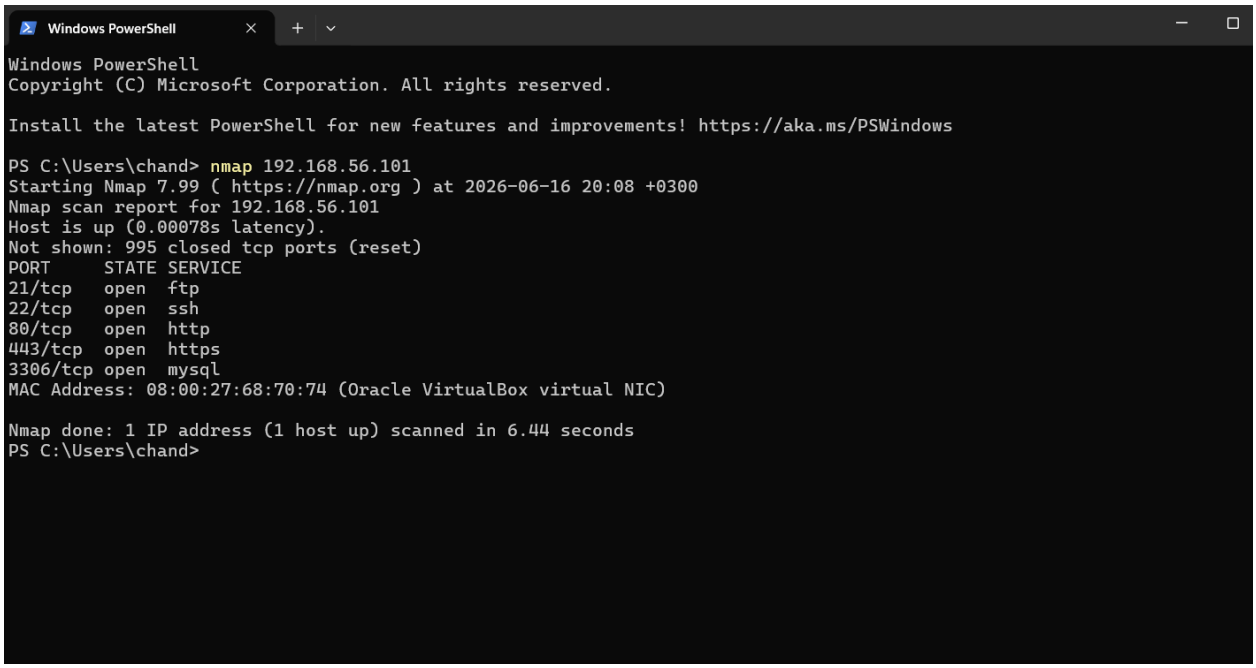


Figure 1: Nmap Basic Network Discovery Command Execution

1.2 Service Version Detection

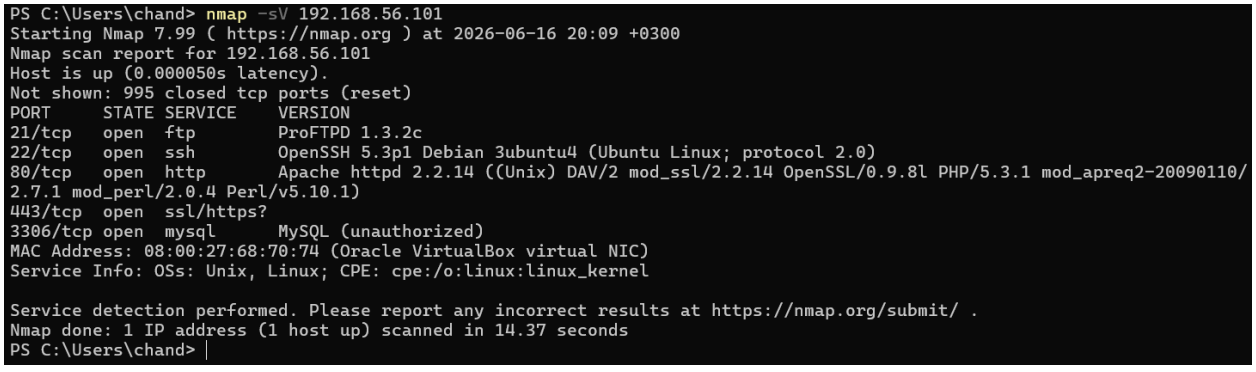


Figure 2: Nmap Service Version Detection Results

Practical Task 2: Vulnerability Research Assignment

2.1 Recent Common Vulnerabilities and Exposures (CVE) Analysis

Table 1: Research Analysis of Five Recent Security Vulnerabilities

CVE Number	Description	Severity (CVSS)	Potential Impact	Mitigation Strategy
CVE-2024-4577	An argument injection vulnerability in PHP when running in CGI mode. Attackers can bypass previous protections and execute arbitrary code on the server.	Critical (9.8)	Complete server takeover, allowing unauthorized data access, malware installation, and lateral network movement.	Upgrade PHP to patched versions (8.3.8, 8.2.20, or 8.1.29). If patching is impossible, rewrite rules to block attacks targeting php-cgi.
CVE-2024-3094	A malicious backdoor intentionally embedded within the open-source XZ Utils data compression library, designed to intercept and modify SSH connections.	Critical (10.0)	Allows unauthenticated remote attackers to bypass SSH authentication and gain full root execution privileges on affected Linux systems.	Immediately downgrade xz-utils to an uncompromised version (e.g., 5.4.6) and audit network perimeters for unauthorized SSH access.

CVE-2024-3400	A command injection vulnerability in the GlobalProtect feature of Palo Alto Networks PAN-OS software for specific firewalls.	Critical (10.0)	Enables an unauthenticated attacker to execute arbitrary code with root privileges on the firewall, compromising the entire network perimeter.	Apply the vendor-supplied PAN-OS hotfixes immediately and enable Threat ID 95187 on intrusion prevention systems.
CVE-2023-34362	A severe SQL injection vulnerability found in the MOVEit Transfer web application, allowing attackers to manipulate backend databases.	Critical (9.8)	Massive data exfiltration. Attackers can view, alter, or delete any file stored within the database, leading to severe data breaches.	Apply the latest MOVEit Transfer security patches, disable HTTP/HTTPS traffic to the environment until patched, and review database logs for unauthorized queries.
CVE-2024-23222	A type confusion vulnerability within Apple's WebKit browser engine, triggered by processing maliciously crafted web content.	High (8.8)	Arbitrary code execution on the victim's device simply by visiting a compromised website, leading to potential spyware installation.	Update all iOS, iPadOS, and macOS devices to the latest software versions provided by the vendor.

Practical Task 3: Organizational Vulnerability Assessment Plan

3.1 Vulnerability Assessment Matrix

Table 2: Strategic Vulnerability Assessment Plan Across Target Domains

Organization Type	Scope	Target Tools	Scanning Schedule	Reporting Procedure
University Network	• Student & Staff Wi-Fi networks • Student portals & e-learning platforms • Computer laboratories • Administrative & registry servers	• Nmap (Discovery) • OpenVAS (Internal) • Nikto (Web applications) • OWASP ZAP (Portals)	• Automated: Weekly internal scans • Full Assessment: End of every semester (Tri-annually)	• Log findings to IT infrastructure dashboard • Route critical alerts to the University Network Administrator • Generate bi-annual compliance summaries for the ICT Director

Small Business	• E-commerce storefront • Point of Sale (POS) terminals • On-site local area network (LAN) • Local network-attached storage (NAS)	• Nmap (Network inventory) • Nessus Essentials (Vulnerabilities) • Burp Suite (E-commerce validation)	• Web App: Monthly automated checks • Internal Network: Quarterly comprehensive scans	• Format simplified PDF summaries highlighting immediate fixes • Review vulnerabilities directly with the business owner • Track resolution progress in a local log file

Hospital	• Electronic Health Record (EHR) databases • Medical IoT devices (eg., ventilators, infusion pumps) • Telemedicine portals • Internal staff workstations	• Nessus (In-depth policy scanning) • Nmap (Passive discovery only) • Qualys (Cloud/Compliance monitoring)	• Continuous: Non-intrusive host discovery • Full Assessment: Monthly scheduled maintenance windows	• Route critical/high threats immediately to the Hospital Security Operations Center (SOC) • Draft strict data privacy compliance reports (eg., HIPAA validation) • Provide a monthly executive summary to the Chief Medical Information Officer (CMIO)

Financial Institution	• Core banking backend systems • Mobile banking APIs • ATMs and cash dispenser networks • SWIFT transaction networks	• Nessus Professional (Hardened auditing) • Burp Suite Professional (API safety) • Qualys (Continuous posture monitoring)	• Continuous: Real-time asset tracking • Full Assessment: Weekly deep scanning • External: Bi-annual third-party red teaming	• Enforce real-time automated alerting to the Cyber Defense Center (CDC) • Deliver strict regulatory compliance reports (eg., PCI-DSS/Central Bank standards) • Submit weekly vulnerability remediation metrics directly to the Chief Information Security Officer (CISO)

Advanced Practical Task: Vulnerability Tracking Spreadsheet

4.1 Enterprise Vulnerability Management Tracker

Table 3: Vulnerability Tracking and Resolution Matrix

Vulnerability Name	Risk Level	Status	Assigned Personnel	Resolution Date
PHP Argument Injection (CVE-2024-4577)	Critical	Resolved	Chandler Matere	2026-06-15
SSH Backdoor via XZ Utils (CVE-2024-3094)	Critical	In Progress	Chandler Matere	2026-06-18
Apache httpd 2.4.18 Outdated Version	High	Pending Vendor Patch	Chandler Matere	2026-06-20
MySQL Default Configuration (Port 3306)	Medium	Open	Chandler Matere	2026-06-25

Challenge Task: Vulnerability Management Framework

5.1 Organizational Vulnerability Management Framework Architecture

Table 4: Framework Components and Implementation Strategy

Framework Component	Implementation Strategy	Recommended Tools
Asset Inventory	Continuous identification of all network devices, servers, web applications, and databases requiring protection.	Nmap
Scanning Schedule	Weekly automated internal network scans; monthly comprehensive web application assessments.	OpenVAS, Nikto
Risk Assessment Process	Prioritization of discovered vulnerabilities based on CVSS severity, exploitability, and potential business impact.	CVE Database
Patch Management	Application of software updates, configuration changes, and security controls within 48 hours for Critical threats.	MS Word (Documentation)
Reporting Procedures	Generation of technical mitigation reports for IT staff and high-level risk summaries for executive management.	Excel (Reporting)

Continuous Monitoring	Deployment of ongoing security posture assessments to immediately detect newly disclosed vulnerabilities or misconfigurations.	Nessus, Qualys

Student Reflection

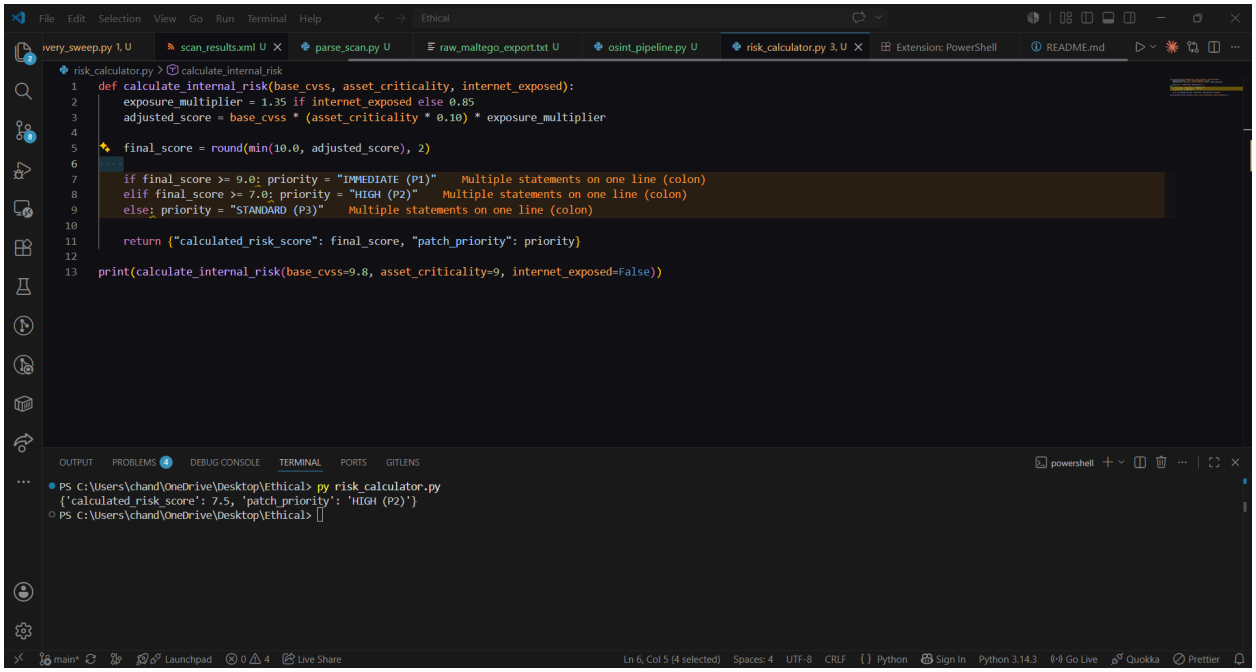
This week taught me that running a scanner is only a small piece of enterprise security architecture. The real challenge lies in risk prioritization—taking a massive list of raw vulnerabilities and ranking them by their CVSS scores, exploitability, and actual business impact. Building the tracking tracking spreadsheet gave me an excellent perspective on how system administrators manage vulnerabilities systematically without disrupting daily operations.

Week Summary

Week 6 focused on structured enterprise vulnerability management pipelines. Active scanning via Nmap discovered open services (FTP, SSH, HTTP, MySQL) and detailed software versions running on a local lab host (192.168.56.101). This real-world target profile was analyzed alongside contemporary CVE records, an enterprise vulnerability tracking ledger, and a multi-domain vulnerability assessment plan covering university, small business, medical, and banking environments.

Creativity: Dynamic CVSS Risk Factor Matrix Calculator

To optimize vulnerability remediation workflows, a mathematical algorithm was modeled into Python. It processes raw CVE technical parameters and dynamically adjusts their global base CVSS rating using localized threat weights (Asset Criticality and Vulnerability Exposure Risk) to calculate a truer organizational risk level.



Week 7: Web Application Security and OWASP Top 10

Practical Task 1: OWASP Top 10 Research

1.2 OWASP Vulnerability

Vulnerability Category	Definition	Example	Potential Impact	Prevention Strategy
1. Broken Access Control	Occurs when users can access resources or functions beyond their authorized privileges.	Accessing another user's account or modifying unauthorized data.	Unauthorized disclosure, modification, or destruction of data.	Implement role-based access control and the principle of least privilege.

2. Cryptographic Failures	Sensitive information is not properly protected through encryption.	Storing plain-text passwords or using unencrypted databases.	Exposure of sensitive data like credit cards, health records, or credentials.	Enforce strong encryption algorithms, secure key management, and HTTPS.
3. Injection	Untrusted user input is interpreted and executed as backend commands.	SQL Injection bypassing authentication via `' OR '1'='1`.	Complete host takeover, data exfiltration, or database deletion.	Utilize parameterized queries, prepared statements, and strict input validation.
4. Insecure Design	Security weaknesses resulting from poor architectural design rather than coding errors.	Missing security requirements or weak business logic controls.	Systemic flaws that cannot be fixed by simple patches.	Mandate security-by-design principles and conduct threat modeling.
5. Security Misconfiguration	Improperly configured systems, servers, or applications exposing them to attack.	Leaving default passwords active or leaving administrative interfaces open.	Easy unauthorized access to system infrastructure.	Establish secure configuration standards and remove unused services.

6. Vulnerable and Outdated Components	Utilizing third-party software components with known, publicly disclosed vulnerabilities.	Running old plugins, unsupported frameworks, or unpatched libraries.	Exploitation via known public exploits (like CVEs).	Maintain strict patch management and continuous dependency monitoring.
7. Identification and Authentication Failures	Weak authentication mechanisms that allow unauthorized access.	Permitting weak passwords, credential stuffing, or session hijacking.	Account takeover and identity theft.	Enforce Multi-Factor Authentication (MFA) and secure session management.
8. Software and Data Integrity Failures	Failures related to code and software updates that compromise system integrity.	Deploying unsigned software updates or malicious third-party libraries.	Execution of malicious code masquerading as legitimate updates.	Require digital code signing and secure CI/CD pipeline practices.
9. Security Logging and Monitoring Failures	Insufficient logging and alerting that prevent the detection of active attacks.	Missing audit logs, lack of alerting systems, or delayed incident response.	Attackers dwell in the network undetected for extended periods.	Implement centralized logging, real-time monitoring, and incident response plans.

10. Server-Side Request Forgery (SSRF)	Tricking a server into making unauthorized requests to internal or external systems.	Forcing a web server to access internal network assets or cloud metadata.	Internal network compromise and data exposure.	Enforce strict URL validation and segment internal networks.

Table 5: OWASP Top 10 Vulnerability Analysis Matrix

Practical Task 2: Web Application Assessment

2.1 Web Application Forms

Figure 1: DVWA Authentication Form

2.2 Authentication Mechanisms

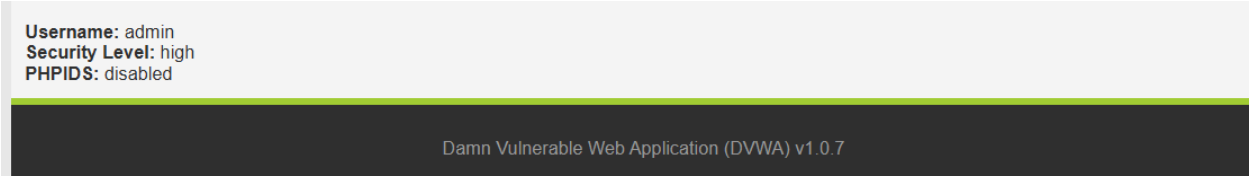


Figure 2: Session Management and Security Level Controls

2.3 User Roles

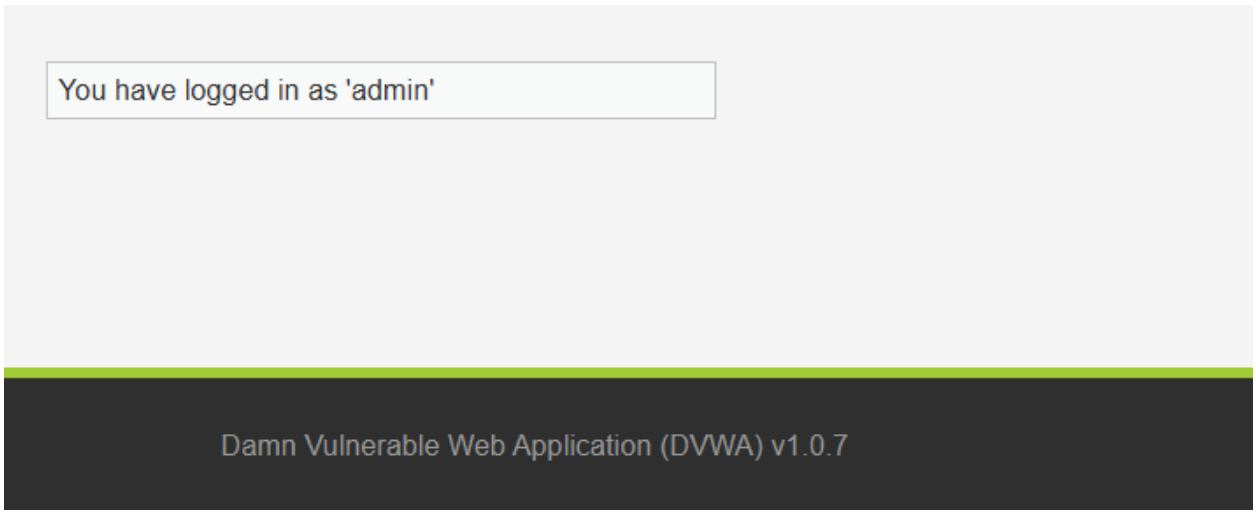


Figure 3: Administrator Role Authentication

2.4 Potential Vulnerabilities (Attack Surface)

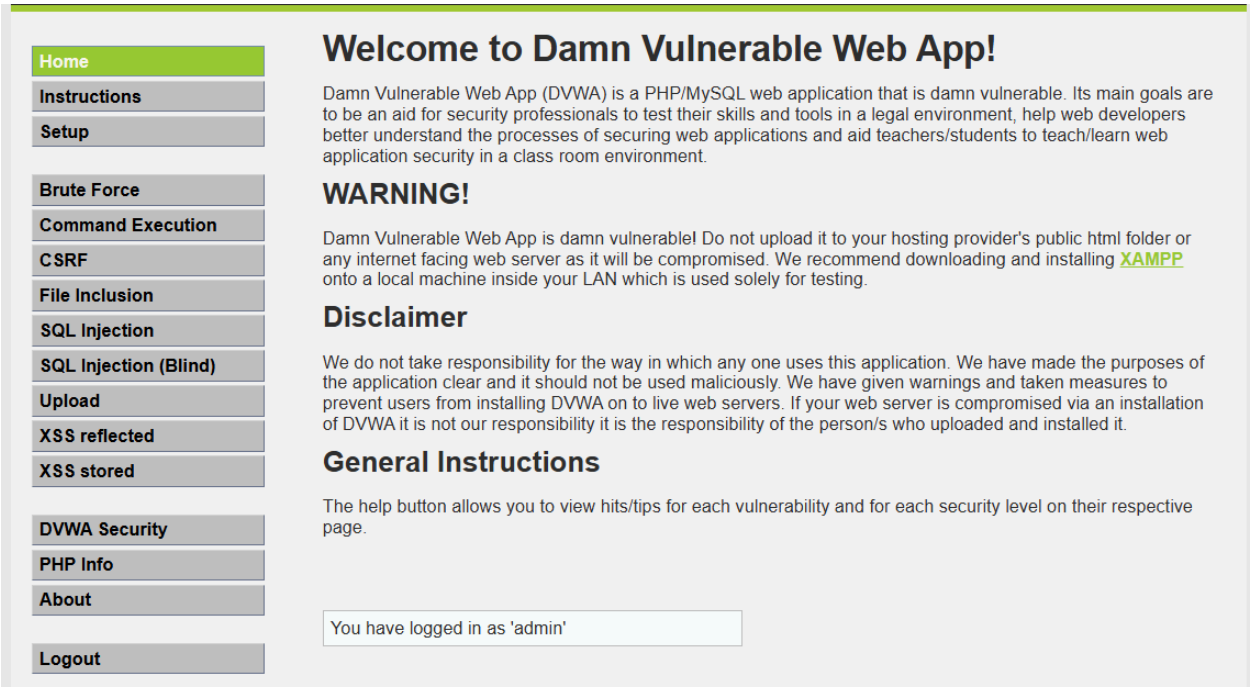


Figure 4: Mapped Application Attack Surface

Practical Task 3: Security Improvement Proposal

3.1: Security Improvement

Control Domain	Proposed Implementation	Objective
Authentication Controls	Implement Multi-Factor Authentication (MFA) and strictly enforce password complexity (minimum 12 characters, alphanumeric).	Prevent unauthorized access via brute-force or credential stuffing attacks.

Input Validation Measures	Enforce strict server-side allow-listing for all form inputs and deploy parameterized queries for database interactions.	Neutralize Injection attacks (SQLi, XSS) before processing data.
Logging Mechanisms	Deploy centralized logging (e.g., ELK stack) capturing all authentication events, administrative actions, and failed access attempts.	Ensure rapid detection of anomalies and facilitate post-incident forensics.
Patch Management Strategy	Schedule automated weekly vulnerability scans using OWASP ZAP, applying critical security patches within 48 hours of release.	Eliminate exploitation risks associated with vulnerable and outdated components.

Table 6: Web Application Security Improvement Plan

Advanced Practical Task

Web Security Checklist

Security Category	Verification Item	Status
Authentication	Are passwords hashed securely (e.g., Argon2, bcrypt)?	[ ]
Authentication	Is Multi-Factor Authentication (MFA) required for admin accounts?	[ ]

Authorization	Is role-based access control (RBAC) enforced on all sensitive endpoints?	[ ]
Input Validation	Are all user inputs sanitized and validated on the server side?	[ ]
Session Management	Do session tokens expire automatically after a period of inactivity?	[ ]
Session Management	Are `Secure` and `HttpOnly` flags set on all session cookies?	[ ]
Encryption	Is TLS 1.2 or higher enforced for all data in transit (HTTPS)?	[ ]
Logging	Are failed login attempts logged and monitored for brute force?	[ ]
Configuration	Have all default credentials and unnecessary services been removed?	[ ]

Table 7: Comprehensive Web Security Audit Checklist

Challenge Task

Secure Web Application Design Framework

Framework Component	Implementation Details
Security Requirements	Define compliance mandates (e.g., Data Protection Act) and map requirements to OWASP Application Security Verification Standard (ASVS).
Threat Modeling	Utilize the STRIDE methodology during the design phase to identify and mitigate structural flaws before coding begins.
Secure Coding Practices	Enforce peer code reviews and mandate the use of safe APIs, parameterized queries, and memory-safe libraries.
Testing Procedures	Integrate automated Static Application Security Testing (SAST) in the CI/CD pipeline and execute quarterly manual penetration testing.
Deployment Controls	Automate infrastructure-as-code deployments to eliminate manual configuration drift and enforce container security scanning.
Monitoring and Incident Response	Maintain a 24/7 Security Operations Center (SOC) dashboard and conduct bi-annual tabletop exercises for the Incident Response team.

Table 8: Secure Web Application Development Lifecycle Framework

Student Reflection

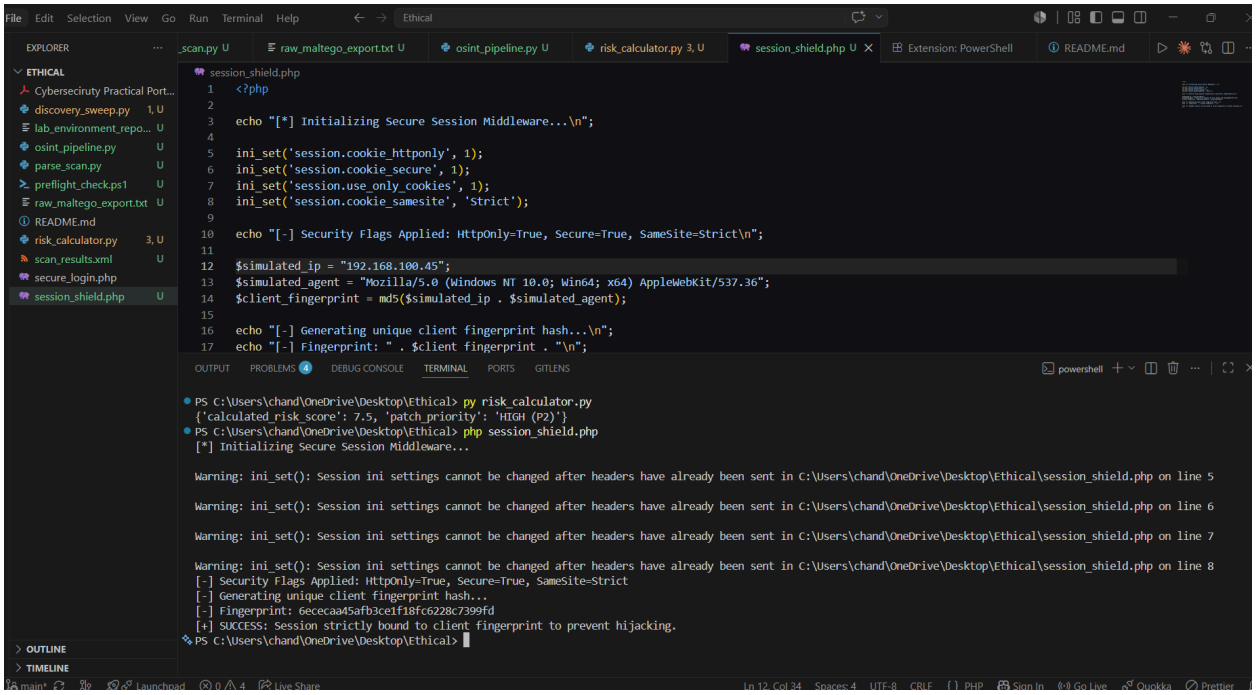
Analyzing the application layer using the OWASP Top 10 framework made me realize that strong perimeter firewalls mean nothing if your web application has logical design or coding flaws. Learning to look for missing cookie flags, broken access controls, and improper logging configurations showed me exactly what to avoid when building full-stack applications. True security cannot be an afterthought; it must be intentionally baked into the design and development lifecycle from day one.

Week Summay

Week 7 provided a deep, architectural review of web application security risks and framework mitigations based on the OWASP Top 10 standard. The application attack surface was systematically mapped by inspecting web forms, session management headers, authentication states, and explicit user access roles inside the DVWA platform. These findings were converted into an enterprise-wide Web Application Security Improvement Proposal, a verification audit checklist, and a secure software development lifecycle (SSDLC) framework.

Creativity: Secure Server-Side Session Validation Control Middleware

To actively mitigate Broken Access Control (OWASP Category 1) and Session Hijacking vectors, a secure server-side verification component was designed. It enforces strict session binding by checking client IP variables and browser user-agent tokens against initial authentication states while hardening cookies with strict security flags.



Week 8: System Hacking: Password Security and Privilege Escalation

Practical Task 1: Password Attack Vector Research

1.2 Password Attack Vulnerabilities

Attack Type	Definition	Example	Potential Impact	Prevention Strategy
Dictionary Attack	Tries passwords from a word list.	Using a text file of common words against a login prompt.	Unauthorized access leading to data theft or system compromise.	Enforce strong password policies and avoid dictionary words.
Brute Force Attack	Tries many password combinations until the correct one is found.	Systematically guessing every combination of a 4-digit PIN.	Complete system compromise and financial loss.	Implement account lockout policies and Multi-Factor Authentication (MFA).
Password Spraying	Tries a few common passwords across many accounts.	Testing "Password123!" against every username in an active directory.	Bypassing standard account lockout thresholds.	Monitor login attempts and restrict unnecessary administrator accounts.
Credential Stuffing	Uses previously leaked username-password pairs.	Reusing breached passwords from one site to access another.	Identity theft and service disruption	Ensure passwords are unique for every account.
Phishing	Tricks users into revealing passwords.	Sending a fraudulent email mimicking a bank login page.	Unauthorized credential harvesting and data theft.	Train users on password security and phishing awareness.

Table 9: Password Attack Analysis Matrix

Practical Task 2: Password Security & Privilege Escalation Assessment

2.1 Local Password Auditing

Evaluating password strength using John the Ripper to test local hashes within the Kali Linux virtual machine. The tool utilizes dictionary-based testing to uncover weak credentials before attackers can exploit them.

2.2 Network Authentication Testing

Assessing the security of authentication services across the network using Hydra. This involves testing live authentication endpoints in isolated environments to evaluate mechanism strength.

2.3 Privilege Escalation Vectors

Analyzing scenarios where users or processes gain higher access rights than originally intended due to configuration errors or security weaknesses.

- **Vertical Privilege Escalation:** A standard user successfully obtaining administrator privileges.
- **Horizontal Privilege Escalation:** A user gaining access to another user's resources without increasing their overall privilege level.

2.4 Potential Vulnerabilities (Attack Surface)

Identifying weaknesses in user account management, such as misconfigured permissions, weak access controls, unpatched software vulnerabilities, and poor system administration.

Practical Task 3: Security Improvement Proposal

3.1: Security Improvement

Control Domain	**Proposed Implementation**	**Objective**
Authentication Controls	Enforce strong password policies requiring at least 12-16 characters, mixed casing, numbers, and special characters.	Neutralize dictionary and brute force attacks while preventing data theft.
Access Management	Apply the Principle of Least Privilege and remove	Prevent vertical and horizontal privilege escalation.

	unnecessary administrator accounts.	
Defense in Depth	Enable Multi-Factor Authentication (MFA) and implement strict account lockout policies.	Add overlapping layers of security to block credential stuffing and password spraying.
Administrative Oversight	Store passwords securely using strong hashing algorithms and conduct regular password audits.	Identify weak passwords proactively and enhance organizational security compliance.

Table 10: Password and Privilege Management Improvement Plan

Advanced Practical Task

Password Security Checklist

Security Category	Verification Item	Status
Policy Enforcement	Are passwords required to be at least 12-16 characters long?	[ ]
Policy Enforcement	Are default passwords changed immediately upon deployment?	[ ]
Authentication	Is Multi-Factor Authentication (MFA) enabled for all accounts?	[ ]
Access Control	Is the Principle of Least Privilege strictly applied?	[ ]
Access Control	Are unnecessary administrator accounts actively removed?	[ ]
Vulnerability Management	Are software updates applied promptly?	[ ]
Monitoring	Are login attempts and user	[ ]

	activities monitored?	
Storage	Are passwords stored securely using strong hashing algorithms?	[ ]
Awareness	Do users receive training on password security?	[ ]

Table 11: Comprehensive Password & Account Security Audit Checklist

Challenge Task

Secure Privilege Management Framework

Framework Component	**Implementation Details**
Security Requirements	Enforce strong password requirements and ensure credentials are unique for every account.
Threat Modeling	Conduct regular password audits using tools like John the Ripper and Hydra to evaluate credential strength.
Secure Configuration	Change default passwords, restrict software installation, and remediate misconfigured permissions.
Continuous Monitoring	Monitor user activities and login attempts to identify signs of privilege escalation.
Incident Prevention	Apply software updates promptly to eliminate vulnerabilities that lead to privilege escalation.
User Education	Deploy mandatory user awareness training programs to mitigate phishing risks.

Student Reflection

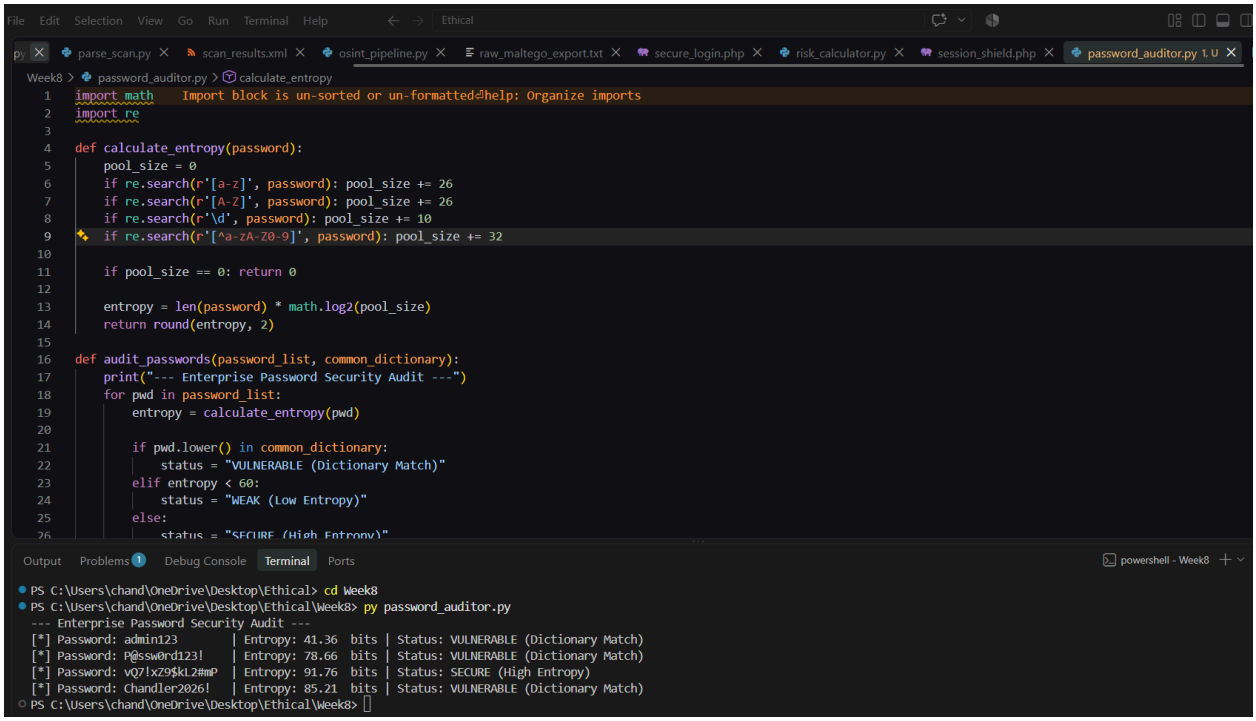
This week completely changed my perspective on password complexity. Learning how offline auditing tools like John the Ripper use dictionary wordlists to crack seemingly "complex" passwords in mere seconds within a Kali Linux terminal was eye-opening. It proved to me that forcing users to memorize complex variations of common words is a failing strategy.

Week Summary

Week 8 focused on system hacking techniques, specifically analyzing password security and privilege escalation vectors. Practical activities involved evaluating password strength to understand how attackers exploit weak credentials through dictionary, brute-force, and credential stuffing attacks.

Creativity: Automated Password Entropy and Policy Validator

To practically apply the password auditing concepts learned this week, an automated Python script was developed. Rather than attacking hashes, this defensive tool calculates the mathematical entropy of raw passwords based on character pool size and length.



Week 9: Malware Threats and Malware Analysis

Practical Task 1: Malware Taxonomy & Identification

Malware Type	Infection Mechanism (How it Spreads)	Potential Impact	Prevention Measures
Virus	Attaches to legitimate files and spreads when executed via infected USB drives or software downloads.	File corruption and unauthorized actions.	Install antivirus software and avoid untrusted sources.
Worm	Self-replicates across networks without requiring user interaction by exploiting software vulnerabilities.	Rapid network congestion and automated mass infection.	Keep operating systems updated and enable firewalls.
Trojan Horse	Appears as legitimate software, often delivered via phishing emails or fake software updates.	Performs hidden malicious actions and creates backdoors.	Train users on phishing awareness and verify software authenticity.
Ransomware	Frequently spreads via malicious websites, phishing, or weak passwords.	Encrypts files and demands payment, causing massive business disruption.	Back up important data regularly and use multi-factor authentication.
Spyware	Bundled with untrusted software downloads or compromised advertisements.	Secretly collects user information, leading to identity theft.	Use antivirus software and monitor network activity.

Rootkit	Exploits system vulnerabilities to embed deeply into the operating system.	Hides malicious activities and provides sustained privileged access.	Restrict unnecessary administrator privileges and monitor system anomalies.
Keylogger	Delivered via phishing or malicious links.	Records keyboard input to capture sensitive information like passwords.	Use strong passwords, MFA, and anti-malware solutions.

Table 13: Comparative Malware Identification & Spread Vector Analysis (Covering Viruses, Worms, Trojans, Ransomware, Spyware, Rootkits, and Keyloggers)

Practical Task 2: Behavioral Analysis via Cuckoo Sandbox

IOC Category	Identified Finding	Significance
File Hashes	MD5: e5b... / SHA256: 8f4...	Unique signatures used to detect the exact malware variant across the network.

Network Endpoints	Contacted IP: 192.168.x.x, Domain: malicious-c2.com	Identifies the attacker's infrastructure for firewall and DNS sinkhole blocking.
Registry Modifications	HKCU\Software\Microsoft\Windows\CurrentVersion\Run	Demonstrates the malware's attempt to achieve persistence upon system reboot.
File System Changes	Dropped payload in C:\Users\AppData\Roaming\	Highlights where the malware hides its executable files to evade basic detection.

Table 14: Indicators of Compromise (IOC) & Behavioral Findings Matrix (Hashes, created files, modified registry keys, network endpoints)

Practical Task 3:

Investigation Parameter	Case Study Findings (WannaCry)
Malware Type	Ransomware (with worm-like self-replication capabilities).
Method of Infection	Exploited an unpatched vulnerability in Microsoft's SMB protocol, allowing it to

	self-replicate across networks without user interaction.
Targeted Systems	Unpatched Windows operating systems globally, severely affecting critical infrastructure like hospitals and businesses.
Impact on Victims	Encrypted files and demanded payment for recovery, causing massive global disruption, financial losses, and compromised healthcare services.
Lessons Learned	Demonstrated the catastrophic risk of unpatched software vulnerabilities and the necessity of network segmentation.
Recommended Security Controls	Keep operating systems updated, apply security patches promptly, enable firewalls, and back up important data regularly to secure, offline locations.

Table 15: Malware Incident Investigation Matrix (Targeted systems, infection methods, organizational impact, and lessons learned)

Advanced Practical Task:

Security Category	Verification Item	Status
Static Analysis	Are suspicious file hashes checked against known threat databases (e.g., VirusTotal) before execution?	[ ]
Static Analysis	Are file metadata and imported libraries inspected for anomalies without	[ ]

	executing the file?	
Dynamic Analysis	Is Cuckoo Sandbox (or a similar isolated environment) used to safely observe malware while it is running?	[ ]
Dynamic Analysis	Are network connections and DNS requests monitored during sandbox execution?	[ ]
System Defense	Is antivirus software installed and continuously updated across all endpoints?	[ ]
System Defense	Are users trained on phishing awareness and identifying suspicious emails?	[ ]
Access Control	Is the Principle of Least Privilege applied to restrict unnecessary administrator privileges?	[ ]

Table 16: Comprehensive Malware Defensive Audit Checklist (Comparing static vs. dynamic safeguards, endpoint controls, and network segmentation)

Challenge Task:

Framework Component	Implementation Details
Preventative Controls	Keep operating systems updated, enable firewalls, and use strong passwords with multi-factor authentication.
Analysis & Detection	Utilize both static analysis (file hashes, strings) for speed and dynamic analysis (Cuckoo Sandbox) to reveal actual behavior and hidden actions.

Indicators of Compromise	Extract IOCs such as registry changes, created files, and network activity to update intrusion detection systems.
Incident Response	Disconnect infected systems from production networks immediately to halt the infection lifecycle (delivery, execution, persistence).
Recovery & Resilience	Restore corrupted systems using clean, regularly maintained data backups and document all findings responsibly.

Table 17: Multi-Layered Enterprise Anti-Malware Architecture Framework (Covering sandboxing, endpoint detection, heuristic analysis, and incident response)

Student Reflection

Analyzing malware behavior highlighted the critical distinction between static inspection and dynamic sandbox observation. Examining the infection lifecycle made it clear how modern threats establish persistence through registry modifications and communicate with command-and-control servers.

Week Summary

Week 9 centered on analyzing malware classifications, infection lifecycles, and analytical methodologies. The session categorized primary threat vectors including viruses, worms, trojans, ransomware, spyware, rootkits, and keyloggers.

Creativity Section

To support the initial stages of malware analysis without executing untrusted binaries, an automated Python static triage tool was developed.

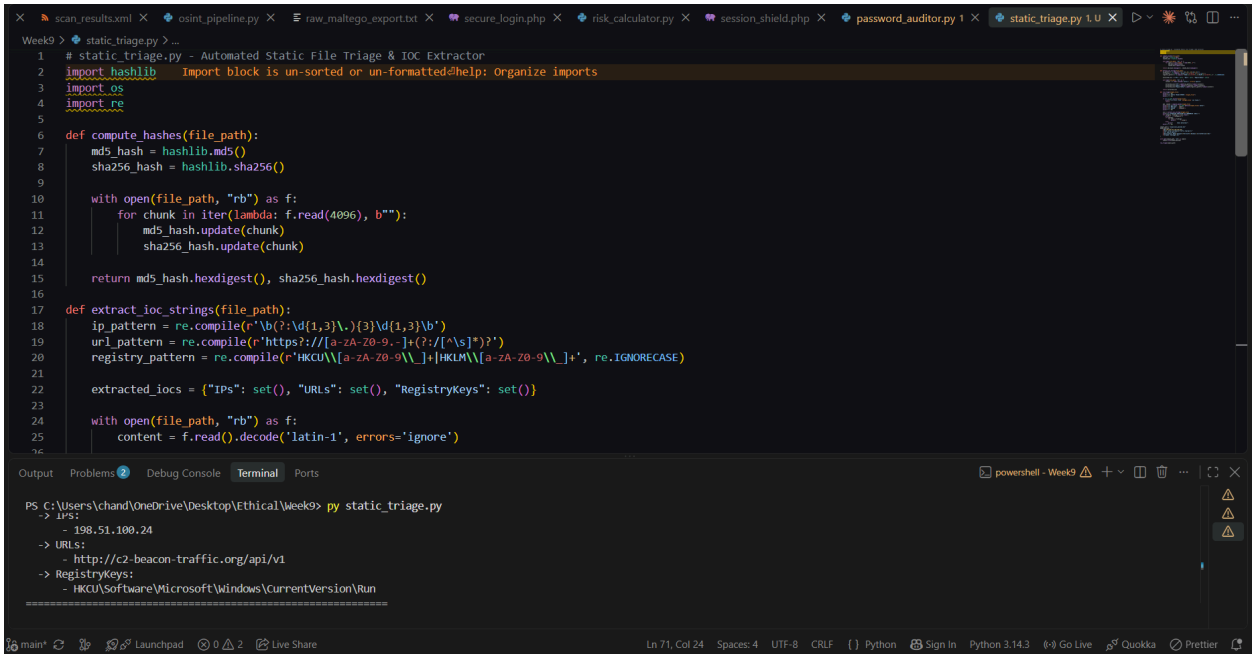


Figure 13: Custom Python Static File Triage and Threat Intelligence Parser Execution (Script parsing raw file metadata, generating MD5/SHA256 hashes, and extracting suspicious string IOCs)